软件学报 ISSN 1000-9825, CODEN RUXUEW
Journal of Software, [doi: 10.13328/j.cnki.jos.000000]
©中国科学院软件研究所版权所有.
E-mail: jos@iscas.ac.cn
http://www.jos.org.cn
Tel: +86-10-62562563

MtAgentLens: 基于 eBPF 的多智能体运行时安全监测与根因定位方法 (面向智能体信息系统的软件新技术)*

唐浩钺†, 刘周康†, 任怡, 张建锋, 赵欣, 马俊, 余杰

(国防科技大学 计算机学院, 湖南 长沙 410073)

通讯作者: 任怡, E-mail: renyi@nudt.edu.cn

摘 要: 大语言模型（Large Language Model，LLM）驱动的多智能体系统通过智能体（Agent）间的任务委派、上下文交接和子智能体生成完成复杂任务。恶意指令可能从用户提示迁移至文件或命令输出等通道，相关行为也可能分散在跨智能体委派、进程和资源访问关系中，使仅分析单条提示或单个智能体上下文的防御难以获得完整的运行时证据。现有防御主要依赖文本或应用层信号，已有基于 eBPF 的智能体可观测方法则侧重系统行为采集与调用关联，尚未充分支持跨智能体委派与资源访问链的联合安全建模和根因定位，也缺少覆盖载荷迁移与多智能体协同攻击的统一评测基准。为此，本文提出载荷迁移分析与攻击场景构造框架 SAFI（Stealthy Attack Framework for Injection），从审查范围迁移、语义包装和跨智能体传播 3 个方面构造 6 种攻击模式，并据此建立包含 86 个冻结场景的 FuxiBench v1.0.0，其中包括 34 个间接注入场景和 23 个多智能体协同攻击场景。在此基础上，本文提出基于 eBPF 的多智能体运行时安全监测与根因定位方法 MtAgentLens。该方法无需修改被监测智能体的业务源码，利用 eBPF 采集 Linux 环境中的进程、文件、网络和 TLS 行为，通过策略边界穿越（Policy Boundary Crossing，PBC）量化敏感资源访问及与跨智能体委派相关联的执行行为，并结合责任图和六维行为评分完成安全判定与因果链回溯。实验结果表明，MtAgentLens 检出了全部 60 个到达操作系统执行阶段的攻击；在 23 个多智能体协同攻击场景中，其真阳性率为 1.000，而 3 种文本基线的真阳性率为 0.087—0.522；在 19 个良性场景中未观察到误报（FPR=0.000，95% CI 为[0.000, 0.158]）。组合消融实验中，加入 eBPF 与 PBC 后，AUROC 由 0.739 提升至 0.953（ΔAUROC=+0.214，$p<0.001$）；3 个多智能体案例的责任链深度分别为 3、5 和 9。单次 SSL 操作的额外时延约为 218 ns，稳态内存占用为 36.9 MB。上述结果表明，对于已产生操作系统级副作用的 Linux 多智能体执行场景，MtAgentLens 能够提供文本和应用层信号之外的运行时证据，并支持基于行为链的安全监测与根因定位。

关键词: 大语言模型；多智能体系统；eBPF；安全监测；根因定位；策略边界穿越；操作系统级可观测性

中图法分类号: TP311

中文引用格式: 唐浩钺, 刘周康, 任怡, 张建锋, 赵欣, 马俊, 余杰. MtAgentLens: 基于 eBPF 的多智能体运行时安全监测与根因定位方法. 软件学报,2026,32(7). http://www.jos.org.cn/1000-9825/0000.htm

英文引用格式: Tang HY, Liu ZK, Ren Y, Zhang JF, Zhao X, Ma J, Yu J. MtAgentLens: An eBPF-Based Method for Runtime Security Monitoring and Root-Cause Localization in LLM-Driven Multi-Agent Systems. Ruan Jian Xue Bao/Journal of Software, 2026 (in Chinese). http://www.jos.org.cn/1000-9825/0000.htm

* 基金项目: 国家重点研发计划(2024YFB4506200); 湖南省联合基金创新研究群体项目(2026JJ10007); 湖南省重点研发计划“面向 AIPC 的智能原生操作系统关键技术研发”

† 唐浩钺与刘周康对本研究贡献相同，列为共同第一作者

MtAgentLens: An eBPF-Based Method for Runtime Security Monitoring and Root-Cause Localization in LLM-Driven Multi-Agent Systems

TANG Hao-Yue†, LIU Zhou-Kang†, REN Yi, ZHANG Jian-Feng, ZHAO Xin, MA Jun, YU Jie

(College of Computer Science and Technology, National University of Defense Technology, Changsha 410073, China)

Abstract: Large language model (LLM)-driven multi-agent systems accomplish complex tasks through inter-agent task delegation, context handoff, and sub-agent spawning. Malicious instructions may migrate from user prompts to channels such as files or command outputs, while the resulting behaviors may be distributed across inter-agent delegation, processes, and resource-access relationships. Consequently, defenses that analyze only a single prompt or the local context of an individual agent may lack the runtime evidence required to reconstruct the complete attack path. Existing defenses primarily rely on textual or application-layer signals, whereas existing eBPF-based approaches to agent observability focus mainly on system-behavior collection and call correlation. These approaches have not been designed to jointly model inter-agent delegation and resource-access chains for security monitoring and root-cause localization. Moreover, a unified benchmark covering both payload migration and multi-agent collaborative attacks remains lacking. To address these limitations, we propose SAFI (Stealthy Attack Framework for Injection), a framework for payload-migration analysis and attack-scenario construction. SAFI defines six attack patterns along three dimensions: inspection-scope migration, semantic wrapping, and cross-agent propagation. Based on SAFI, we establish FuxiBench v1.0.0, a frozen benchmark comprising 86 scenarios, including 34 indirect-injection scenarios and 23 multi-agent collaborative attack scenarios. We further propose MtAgentLens, an eBPF-based method for runtime security monitoring and root-cause localization in multi-agent systems. Without modifying the application source code of the monitored agents, MtAgentLens uses eBPF to collect process, file, network, and TLS behaviors in Linux environments. It uses Policy Boundary Crossing (PBC) to quantify sensitive-resource accesses and execution behaviors associated with inter-agent delegation, and combines a responsibility graph with six-dimensional behavioral scoring to support security decision-making and causal-chain tracing. Experimental results show that MtAgentLens detects all 60 attacks that reached the operating-system execution stage. Across 23 multi-agent collaborative attack scenarios, it achieves a true-positive rate of 1.000, whereas three text-based baselines achieve true-positive rates ranging from 0.087 to 0.522. No false positives are observed in 19 benign scenarios (FPR = 0.000, 95% CI: [0.000, 0.158]). In a combined ablation study, adding eBPF and PBC improves AUROC from 0.739 to 0.953 (ΔAUROC = +0.214, $p < 0.001$). The responsibility chains reconstructed in three multi-agent case studies have depths of 3, 5, and 9, respectively. The additional latency per SSL operation is approximately 218 ns, and steady-state memory consumption is 36.9 MB. These results indicate that, for Linux multi-agent executions that produce operating-system-level side effects, MtAgentLens provides runtime evidence complementary to textual and application-layer signals and supports behavior-chain-based security monitoring and root-cause localization.

Key words: LLM; multi-agent system; eBPF; security monitoring; root cause localization; policy boundary crossing; operating-system-level observability

大语言模型（LLM）驱动的多智能体系统通过任务委派、上下文交接和子智能体生成等机制组织复杂工作流，已逐步应用于代码评审、根因分析和 AADL 模型生成等任务[1]。以 LangGraph 为代表的编排框架进一步支持多个智能体（Agent）之间的分工与协作[2]。与此同时，系统的安全边界也由单一提示文本扩展至智能体间通信、工具执行以及进程、文件和网络资源访问等多个环节。

现有智能体安全机制主要依据文本或应用层信息进行判定，能够识别提示注入、有害内容和异常工具调用，但难以持续关联执行过程中产生的异构行为证据。当任务涉及文件读取、进程派生、网络连接和跨智能体交接时，与攻击相关的信息可能分布在不同数据通道和协作阶段，仅分析单条提示文本或单个智能体上下文，难以完整还原攻击的传播路径及其运行时影响。

为系统刻画并复现其中的载荷通道迁移现象，本文提出载荷迁移分析与攻击场景构造框架 SAFI（Stealthy Attack Framework for Injection）。SAFI 将用户提示、工具调用参数、文件内容和 Shell 命令输出建模为审查强度不同的数据通道。在本文考察的防御配置中，用户提示通常接受较严格的文本检查，而文件内容和命令输出缺少同等强度的审查，由此形成载荷由提示通道向文件或输出通道迁移的条件。SAFI 进一步从审查范围迁移、语义包装和跨智能体传播 3 个维度构造 6 种攻击模式，并用于生成 FuxiBench v1.0.0 中的 34 个间接注

入（Type B）场景。因此，SAFI 在本文中承担威胁分析与场景构造功能，为后续运行时监测方法的设计和评估提供攻击模型与数据基础。

除载荷通道迁移外，多智能体协同还引入了交互拓扑层面的证据缺口。在多智能体协同攻击（Type C）中，单个智能体的提示文本和局部行为可以不包含显式恶意语义，而攻击目标通过委派链、共谋环和角色劫持等关系逐步形成。以单个提示文本或单个智能体上下文为分析单元的检测器难以完整获得这些跨智能体关系。在本文的冻结评测口径下，AgentSentinel[3]在 23 个 Type C 场景上的真阳性率为 0.130。UDora[4]和 ChatInject[5]分别揭示了动态工具调用观测值和会话模板结构所引入的注入风险，但其研究对象仍以单智能体上下文中的载荷注入为主。上述结果表明，既有方法能够刻画局部上下文中的攻击载荷，但对跨智能体交互拓扑的覆盖仍然有限。

围绕上述风险，现有防御主要沿提示级检测、安全对齐训练和运行时监测三条技术路径发展。提示级检测方法通过文本分类识别注入与有害内容，代表性工作包括 LlamaFirewall[6]、Prompt Guard[7]、LLM Guard[8]和 WildGuard[9]；安全对齐方法则利用结构化输入或偏好优化增强模型对注入指令的鲁棒性，如 StruQ[10]、SecAlign[11]和 Meta SecAlign[12]。这些方法主要作用于指令进入执行流程之前。运行时防御方法进一步分析智能体的工具调用、执行事件或交互图，包括 AgentSentinel[3]、MAStitch[13]、BlindGuard[14]和 GUARDIAN[15]，但其观测信息仍主要来自应用层，尚未统一关联进程、文件、网络和 TLS 行为。相关综述也表明，现有研究已对越狱攻击和对抗性攻击形成较系统的分类[16,17]，但面向载荷跨通道迁移及跨智能体拓扑传播的运行时证据组织仍有待完善。

针对上述问题，本文提出 MtAgentLens，一种基于扩展伯克利包过滤器（extended Berkeley Packet Filter，eBPF）的多智能体运行时安全监测与根因定位方法。该方法在不修改智能体业务源码的条件下采集 Linux 环境中的进程、文件、网络和 TLS 行为，并将敏感资源访问与跨智能体委派表示为策略边界穿越（Policy Boundary Crossing，PBC）。在此基础上，MtAgentLens 通过责任图关联智能体身份、资源访问、上下文传播和委派关系，再结合六维行为评分完成安全判定与因果链回溯。SAFI 用于描述和构造攻击场景，MtAgentLens 用于采集并分析运行时证据，二者分别对应威胁建模与防御方法。

在方法论上，MtAgentLens 借鉴智能运维（Artificial Intelligence for IT Operations，AIOps）中多源事件关联与传播路径回溯的基本思路[19-23]，并将其迁移至多智能体运行时安全场景。系统采用三层流水线（图 1）：第一层采集进程、文件、网络和 TLS 事件；第二层完成身份解析、PBC 量化与责任图构建；第三层利用六维行为评分和 Observer 语义校准给出安全判定。该结构使行为采集、证据组织与风险判断相互分离，从而同时支持在线监测与事后调查。

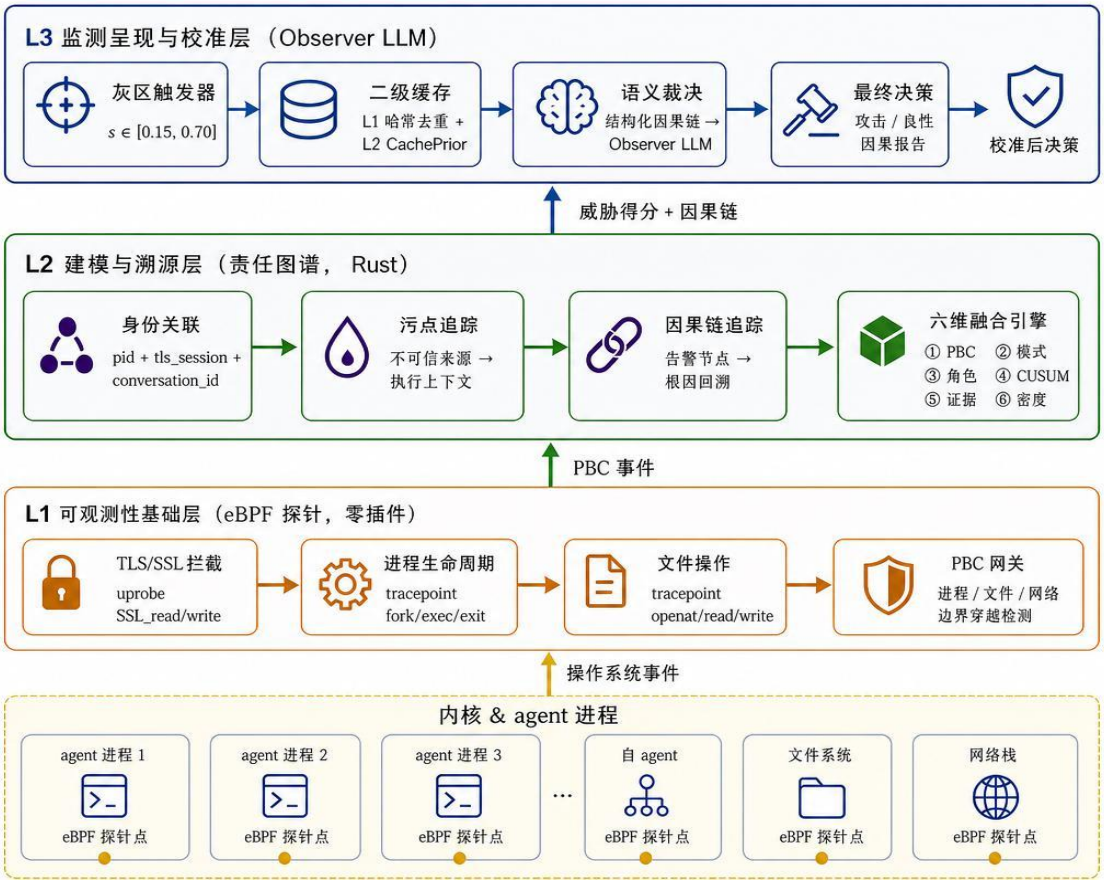


图 1　MtAgentLens 三层流水线架构

为评估 MtAgentLens，本文围绕四个研究问题展开：RQ1，该方法对直接注入、间接注入和多智能体协同攻击的检测能力如何，责任图能否支持案例级根因定位；RQ2，eBPF、PBC、责任图和 Observer 等组件分别产生何种贡献；RQ3，该方法在外部基准 InjecAgent[24]上的泛化表现及退化边界如何；RQ4，持续监测所产生的运行时开销是否处于可接受范围。

本文的主要贡献如下。（C1）提出 SAFI 载荷迁移分析与攻击场景构造框架，从 3 个维度形成 6 种攻击模式，并据此构建 34 个 Type B 场景。（C2）建立 Type C 多智能体协同攻击模型，以 23 个场景刻画智能体团队毒化、子智能体生成和隐身团队协作等跨智能体攻击路径。（C3）设计并实现 MtAgentLens，通过 eBPF 事件采集、PBC 量化、责任图和六维行为评分实现多智能体运行时安全监测与根因定位。（C4）构建并冻结包含 86 个场景的 FuxiBench v1.0.0；实验中，MtAgentLens 检出了全部 60 个到达操作系统执行阶段的攻击，在 23 个 Type C 场景上取得 1.000 的真阳性率，且在 19 个良性场景中未观察到误报。（C5）通过组合消融、案例研究和性能实验分析各层作用：加入 eBPF 与 PBC 后 AUROC 由 0.739 提升至 0.953，3 个多智能体案例的责任链深度分别为 3、5 和 9，系统稳态内存占用为 36.9 MB。

论文其余部分组织如下。第 1 节回顾相关研究；第 2 节给出威胁模型、攻击分类与 SAFI 框架；第 3 节介绍 MtAgentLens 的系统设计；第 4 节说明关键算法；第 5 节和第 6 节分别给出实验设置与评估结果；第 7 节讨论方法的适用边界与后续研究方向。

1 相关工作

与 MtAgentLens 直接相关的研究主要分布在 LLM 智能体安全、eBPF 可观测性、多智能体监测与 AIOps 根因定位三个方向。前两类工作分别提供了攻击面、防御机制和系统级观测基础，后一类工作提供了多源事件关联、异常检测和因果链回溯的方法参照。综合来看，已有研究已经能够覆盖 prompt 注入、越狱攻击、工具调用滥用、应用层运行时防御和系统调用级观测等问题，但面向跨智能体委派、上下文交接和资源访问链的操作系统级安全监测仍需要进一步建模。

1.1 LLM智能体安全：从攻击面到防御谱系

LLM 智能体攻击研究首先从 prompt 注入和越狱攻击展开。AutoDAN[25]、TAP[26]和 GCG[27]分别利用遗传搜索、攻击树搜索和梯度搜索生成高成功率攻击提示，说明单轮文本输入本身可以构成有效攻击面。随着 LLM 应用从问答系统发展为工具增强型 Agent，攻击载体进一步扩展到工具返回内容、文件输入和模板结构。InjecAgent[24]对间接 prompt 注入进行系统评估，揭示外部数据中的恶意指令可以通过工具链影响 Agent 行为；UDora[4]和 ChatInject[5]则分别讨论动态工具调用劫持和 chat template 滥用。AgentHarm[28]从危害类别角度扩展了 Agent 安全评测，Formalizing[29]所提出的 LLM-Integrated Application 抽象，则为区分 instruction、untrusted data 和执行响应提供了基础。国内研究中，台建玮等[16]和李南等[17]对 LLM 对抗性攻击与越狱攻击进行了综述，王路桥等[1]则从多智能体协作代码评审的应用场景展示了 Agent 工程化落地形态。

多 Agent 场景进一步改变了攻击的组织方式。SoK[30]从整体攻防图景梳理了 LLM 智能体安全问题，Disguise While Defense [31] 将攻防过程放入多 Agent 博弈，Propagating Unsafe Actions[32]则指出具身智能体中的不安全动作可能在多轮决策中传播。与单 Agent 攻击相比，多 Agent 攻击的判定对象不再只是单条 prompt 或单次工具调用，还包括委派链、角色分工、上下文交接和共识关系。该变化构成本文 Type C 攻击分类的直接问题背景。

防御方法大致形成了三条技术路线。第一类是 prompt 级检测，以 LlamaFirewall[6]、Prompt Guard[7]、LLM Guard[8]和 WildGuard[9]为代表，其核心是对输入或输出文本进行风险分类。这类方法部署成本低，适合直接注入、越狱文本和有害内容生成检测，但对文件读取后才暴露的载荷、进程级副作用和跨 Agent 拓扑关系掌握有限。第二类是安全对齐与输入结构化方法，StruQ[10]、SecAlign[11]和 Meta SecAlign[12]通过结构化指令边界或偏好优化增强模型对注入的鲁棒性；Caging the Agents[33]从能力约束出发限制 Agent 可执行动作，UniGuardian[34]尝试构建统一防护框架。该路线强调执行前约束，适用于降低攻击进入工作流的概率。第三类是运行时防御，Agent Sentinel[3]、MAStitch[13]、BlindGuard[14]和 GUARDIAN[15]将监测推进到 Agent 执行阶段，其中 GUARDIAN 的时序图思路与本文责任图具有相近的问题意识。上述方法为 Agent 安全提供了重要基础，但多数仍以文本、API 日志或单 Agent 上下文为主要分析对象。

评测与补充研究进一步丰富了该领域。AgentDojo[35]、EnIGMA[36]、LLM Strategically Deceive[37]、Not What You Signed Up For[38]和 SmoothLLM[39]分别从基准环境、交互式任务、策略性欺骗、用户意图偏离和扰动平滑等角度刻画 Agent 风险。它们共同说明，LLM 智能体安全已经从单次文本分类问题扩展为覆盖任务环境、工具链和行为后果的系统问题。本文的差异在于，将监测对象落到 OS 级行为事件，并进一步把这些事件组织为跨 Agent 可回溯结构。

1.2 eBPF可观测性与安全监控

eBPF（extended Berkeley Packet Filter[18]通过 kprobe、tracepoint 和 uprobe 在内核态运行受限程序，能够在较低侵入条件下采集系统调用、进程、网络和用户态函数事件。在云原生安全监控中，Tetragon[40]、Falco[41]等云原生安全工具已经证明 eBPF 可用于运行时审计、策略执行和威胁检测。AgentSight[42]将 eBPF 引入 LLM

智能体可观测性，通过 uprobe 捕获 SSL/TLS 明文片段，并把 LLM API 调用与系统调用进行关联，说明 eBPF 可以进入智能体运行时观测链路。

与传统主机审计相比，LLM 智能体场景对 eBPF 观测提出了新的组织需求：单个系统调用只说明“发生了什么”，而安全判定还需要知道该事件由哪个 Agent 发起、是否由上游委派触发、是否与文件载荷或上下文传播相关。MtAgentLens 因此不只采集 TLS、进程、文件和网络信号，还将这些信号映射到 Agent 身份、策略边界和责任图边。该定位不同于以性能分析和调用关联为主的 AgentSight[42]，也不同于通用容器运行时检测工具。

eBPF 运行环境本身也需要受到约束。Moat[43]、Hive[44]和 KrakenGuard[45]分别从隔离、访问控制和内核监控安全角度讨论了 eBPF 基础设施的防护，Cross Container Attacks[46]则指出云环境中的 eBPF 可能引入跨容器攻击面。这些研究提示，基于 eBPF 的智能体安全监测需要清楚界定可信计算基和攻击者能力。本文在威胁模型中将操作系统内核与 eBPF 观测基础设施作为 TCB，不讨论内核级攻击者对观测平面的直接破坏。

1.3 多智能体系统监测与AIOps方法论迁移

多智能体系统监测研究关注多主体协作中的故障、异常和责任归属。AgenTracer[47]通过反事实重放和故障注入支持多智能体系统的事后故障归属，AgentTuning[48]从监督微调角度说明 Agent 能力可以被定向塑造，Kazari 等[49]在合作式多智能体强化学习场景下讨论去中心化异常检测，Electrode[50]则展示了 eBPF 对分布式协议观测与加速的系统价值。这些工作与本文共同关注多主体系统中的行为关联，但本文的对象是 LLM 智能体安全事件，重点不在一般故障归属，而在攻击载荷、敏感资源访问和跨 Agent 传播关系的联合刻画。

AIOps 和根因定位研究为本文提供了方法论参照。AmazeMap[19]以多层次影响图组织系统组件关系，于庆洋等[20]利用调用链控制流进行异常检测；PASER[21]通过多维 KPI 分解分析异常来源，SOTIMiner[22]从时序不变式中发现异常模式，BARO[23]使用多元贝叶斯变点检测增强异常识别的鲁棒性。RCA Causal[51]与 Chain-of-Event[52]则从因果结构和事件链角度支撑根因回溯。国内相关工作中，李晓卓[53]、包航宇[54]、殷康璘[55]和张明悦[56]分别从软件工程、AIOps、系统韧性和多智能体博弈等方向提供了补充背景。

上述研究表明，多源观测、图结构建模和因果链回溯已经在智能运维和分布式系统分析中形成成熟思路。将其迁移到 LLM 多智能体安全场景时，关键差异在于：监测对象由服务调用和 KPI 异常转为 prompt、文件、进程、网络和 Agent 委派关系；分析目标由故障定位转为攻击检测与责任归因；证据结构由服务拓扑转为 Agent-操作系统交互图。表 1 概括了相关技术路线的适用边界与本文定位。

表 1 相关工作技术路线、适用边界与本文定位

技术路线	代表工作	主要信号	适用边界	与本文关系
Prompt 级检测	LLM Guard[8]、WildGuard[9]	输入/输出文本	直接注入、越狱文本、有害内容生成	提供执行前文本侧基线
安全对齐与结构化输入	StruQ[10]、SecAlign[11]、Meta SecAlign[12]	指令边界、偏好数据	降低注入进入模型决策的概率	与 OS 级运行时监测互补
智能体运行时防御	Agent Sentinel[3]、GUARDIAN[15]	智能体交互日志、时序图	单 Agent 或应用层运行时审计	与责任图思想接近，但观测层级不同
eBPF 可观测性	AgentSight[42]、Electrode[50]	TLS、系统调用、协议事件	运行时低侵入观测	提供 OS 级信号采集基础
AIOps 根因定位	AmazeMap[19]、PASER[21]、SOTIMiner[22]、BARO[23]、Chain-of-Event[52]	多维指标、影响图、事件链	分布式系统异常检测与溯源	支撑多源融合和因果链回溯

总体而言，现有研究已经在文本级防御、对齐训练、运行时审计和系统可观测性方面形成了较完整的技术谱系。MtAgentLens 的研究位置在于，把 OS 级观测作为跨 Agent 工作流的公共证据平面，并用 PBC、责任图和六维评分将进程、文件、网络与 TLS 行为组织为可检测、可回溯的安全信号。

2 威胁模型

MtAgentLens 的威胁模型由执行轨迹、攻击者能力、可信边界、攻击类型和载荷迁移框架构成。该模型把 prompt、文件、智能体间消息和 OS 事件放入同一分析对象中，为后续 PBC、责任图和六维评分提供统一输入。

2.1 攻击者能力与边界

我们假设攻击者具备以下能力：（1）可向 Agent 系统提交任务（prompt 或文件）；（2）可控制多智能体协作中的部分 Agent（拜占庭行为）；（3）可建立外部 C2 连接；（4）可在文件系统中放置任意内容；（5）可发起网络连接以泄露数据。

本文将操作系统内核与 eBPF 观测基础设施作为可信计算基（TCB），不讨论内核级攻击者或针对 eBPF 运行环境本身的规避行为。攻击者不具备直接修改内核、直接读取 Agent 进程内存或控制监测基础设施的能力；其影响 Agent 行为的主要路径是 prompt、文件、智能体间交互和工具执行。LLM 安全过滤器可能在 OS 执行前阻止部分攻击，实验中将这类场景计入 Intent-to-Treat 统计口径（ITT-TPR 上限=60/67,7 个攻击被 LLM 过滤器阻挡）。

2.2 执行轨迹与监测问题定义

定义 1（多智能体执行轨迹）给定一次多智能体任务执行，其轨迹记为 $\tau=(A,R,E_{os},E_{msg},T)$。其中，$A=\{a_1,...,a_m\}$为参与执行的 Agent 集合，R 为文件、网络端点、凭据、进程和工具调用等资源集合；E_{os} 为操作系统可观测事件集合，每个事件可表示为 $e=(p,op,r,t,\eta)$，分别对应进程、操作、资源、时间戳和证据属性；E_{msg} 为智能体间消息、委派和上下文交接事件集合；T 为事件偏序关系，用于刻画执行中的先后约束。本文的 OS 级监测主要依赖 E_{os}，并通过 TLS 明文片段、进程树和工具调用参数推断 E_{msg} 中可观测的部分。

定义 2（操作系统可观测攻击）设 B 为策略边界集合，包含敏感文件、外部网络端点、凭据访问、跨 Agent 委派和上下文传播规则。若一次攻击在执行轨迹 τ 中产生至少一个与攻击目标相关的 OS 级副作用，且该副作用能够映射为 E_{os} 中的进程、文件、网络、TLS 或工具调用事件，则称该攻击为 OS 可观测攻击。被模型安全过滤器在执行前阻断、且未产生 OS 事件的攻击，不属于 OS 级监测的可观测范围。

基于上述定义，本文研究的问题可表述为两个函数：安全判定函数 $f(\tau,B)\rightarrow y$，其中 $y\in\{BENIGN,ATTACK\}$；溯源函数 $g(\tau,B)\rightarrow\langle a_0,e_1,a_1,...,e_k,r_k\rangle$，输出由上游 Agent、关键事件、下游 Agent 与目标资源构成的责任链。问题约束包括：（1）不修改 Agent 业务代码；（2）不假设特定 Agent 框架暴露完整内部状态；（3）以 OS 级事件和可推断的 Agent 间交互作为主要证据；（4）对纯文本有害内容仅提供互补而非替代性的防护。

2.3 三类操作型攻击：按载荷载体分类

基于 Formalizing[29]提出的 LLM-Integrated Application 模型（instruction prompt+untrusted data→LLM→response），我们将 LLM 智能体攻击按载荷载体分为互斥的三类（表 2）。

定义 3（载荷载体与攻击类型）设 c(x)表示攻击载荷或恶意意图首次进入执行轨迹的载体。若 c(x)为用户 prompt 文本，则归为 Type A；若 c(x)为随任务提交或工具读取的文件内容，则归为 Type B；若恶意性主要由 Agent 间委派、上下文交接、共识操纵或子 Agent 生成关系形成，而单条 prompt 或单个文件不足以完整表达攻击目标，则归为 Type C。三类攻击以首要载体和判定所需上下文为分类依据；当场景同时包含多个载体时，以形成攻击语义所必需的最高阶载体为准。

表 2 三类操作型攻击按载荷载体分类

类型	攻击向量	载荷载体	prompt 可见性	攻击场景数	文本级检测可行性
Type A—直接注入	攻击者 prompt 文本	用户 prompt	可见	10	可由文本级方法处理,但存在漏报
Type B—间接注入 (SAFI)	随附文件	夹具文件	文件读取前不可见	34	需要文件或工具读取证据
Type C —多智能体协同	智能体间交互拓扑	委派/上下文/共识关系	单条 prompt 不充分	23	需要跨 Agent 拓扑证据

Type A（直接 prompt 注入）以 AgentHarm[28]（1,100 测试用例,11 危害类别,ICLR 2025）为基准代表。MtAgentLens 将 Type A 作为基准对照，用于验证 OS 级信号对 prompt 文本攻击的补充检测能力。

Type B（间接注入/SAFI）关注载荷从 prompt 通道迁移到随任务提交的夹具文件（fixture files）后的检测问题。此时 prompt 文本描述合法开发任务，但文件内容携带恶意载荷。InjecAgent[24]、UDora[4]和 ChatInject[5]从不同角度展示了间接注入、工具调用劫持和模板结构滥用等攻击向量。

Type C（多智能体协同）是本文重点关注的跨智能体攻击类别，包含 5 个子类别共 23 个具体攻击场景。第一类为智能体团队毒化（15 场景），涵盖共谋环、共识操纵、角色劫持、渐进式角色偏移和恶意反馈环。第二类为子智能体生成（5 场景），涵盖影子智能体、委派链、上下文多跳和碎片化载荷。第三类为隐身团队协作（3 场景），涵盖隐身编译协调、隐身值班交接和隐身迭代规划。Type C 攻击的核心特征在于：单个 Agent 的 prompt 和局部行为可以保持良性，而恶意性主要体现为 Agent 之间的委派、传播或共识关系。这使仅分析单个 prompt 的文本级检测器面临明显的信息缺口。

2.4 SAFI框架：载荷迁移与安全不对称性

SAFI（Stealthy Attack Framework for Injection）在本文中作为基准构造与载荷迁移分析框架，用于刻画 LLM 智能体工作流中不同数据通道的审查强度差异。表 3 总结了本文实验环境下观察到的四级审查边界不对称性，其中典型拒绝率由 SAFI 构造阶段的预实验日志统计得到。

表 3 SAFI 框架：四级审查边界不对称性

数据通道	审查强度	典型拒绝率	审查主体
用户 prompt 文本	深度扫描	30-80%	LLM 安全过滤器
工具调用参数	部分扫描	10-30%	框架输入验证
文件内容（工具读取）	较少显式扫描	~0%	通常未显式配置
Bash 命令输出	较少显式扫描	0%	通常未显式配置

SAFI 使用三类载荷迁移策略刻画上述不对称性：（1）审查范围迁移，将载荷从 prompt 通道移至文件或输出通道；（2）语义包装，通过语言转换（P5 CC-BOS 文言文绕过）或语义拆分（P6 载荷拆分）降低单

步文本审查的可见性；（3）跨 Agent 传播，利用多智能体工作流的时间展开将攻击影响分散到多个合法步骤中。

SAFI 包含 6 种具体攻击模式（P1-P6），覆盖文件注入、配置伪装、构建验证、代码生成、语言包装和载荷拆分等形式。其作用在于将 Formalizing 的两通道模型（instruction+data）扩展为更细粒度的审查边界模型，说明文本级防御在 Type B/C 攻击上需要额外的文件、工具调用或跨 Agent 上下文证据。

补充说明：SAFI P5 与 P6 攻击模式实验口径

P5（文言文绕过）使用 CC-BOS 语言转换策略将英文恶意指令翻译为文言文后嵌入任务描述。在本文实验环境中，该策略的攻击成功率约为 80%，高于直接英文注入设置。P6（载荷拆分）将攻击指令拆分为多个语义碎片，分散嵌入任务的多个步骤中，利用多 Agent 工作流的时间展开降低单步审核可见性；在本文实验环境中，其攻击成功率约为 71%。两种模式的完整 prompt 模板和实验配置随 SAFI 场景材料提供。

2.5 检测范围说明

MtAgentLens 的检测范围界定为：覆盖产生 OS 级副作用（进程、文件、网络、TLS 事件）的攻击。被 LLM 安全过滤器在 OS 执行前阻挡的攻击（7/67）不在 OS 级监测范围内，ITT-TPR 统计上限=60/67。MtAgentLens 不检测纯文本层面的有害内容（无 OS 副作用），也不替代 LLM 安全过滤器；两者在执行前文本审查与执行后行为监测上形成互补关系。

此外，FuxiBench 的良性场景规模（n=19）限制了 FPR 估计精度（95% CI:[0.000,0.158]）。更大规模的良性场景验证（如 100+工业场景）是未来工作的重要方向。

3 系统设计

3.1 设计动机：智能体-操作系统关系分析

MtAgentLens 的设计出发点是 Agent 行为与 OS 事件之间的映射关系。LLM 推理通常伴随 SSL/TLS 通信，工具调用会产生进程创建或命令执行，文件读写会触发 VFS 操作，敏感文件或外部网络资源访问则可能形成策略边界穿越（PBC）。这些 OS 级事件与文本内容具有互补性：即使 prompt 表达保持良性，只要任务执行经过进程、文件、网络或 TLS 通道，系统仍可获得文本层之外的行为证据。MtAgentLens 因此将 OS 层作为跨 Agent 工作流的公共监测平面。

从 Agent-操作系统映射关系出发，本文抽象出两个互补的身份信号：（1）root_pid，用于表示进程层级关系，捕获 Agent 的创建、委派和退出；（2）tls_session，用于表示 API 会话，将 TLS 明文事件与 Agent 身份关联。第 3.4.2 节进一步说明如何通过 UnionFind 融合这两类信号，并在子进程拥有独立 TLS 会话时识别子 Agent 边界。

3.2 设计原则与总体架构

MtAgentLens 遵循四项设计原则：（1）零插桩，不修改 Agent 业务代码，监测主要通过 eBPF 内核探针实现；（2）框架弱依赖，采集层不绑定 LangGraph 等特定编排框架，而以进程、文件、网络和 TLS 等操作系统级信号作为统一入口；（3）行为证据优先，检测主要依据运行时行为信号，文本语义分析仅作为辅助校准手段；（4）校准解耦，检测决策与语义评估分层，降低 LLM 仲裁不确定性对基础信号通道的影响。

总体架构为三层流水线（图 1）。L1（可观测性基础层）通过 TLS/SSL、进程、文件和网络探针采集 OS 级事件；L2（建模与溯源层）完成身份解析、PBC 量化、责任图构建、污点追踪和因果链回溯；L3（检测与校准层）通过六维行为评分体系给出安全判定，并仅在灰色地带触发 Observer 语义仲裁。三层之间形成从事件采集、证据组织到风险判定的递进关系。

3.3 L1：操作系统级信号采集

3.3.1 eBPF 探针体系

MtAgentLens 的 L1 层部署四类 eBPF 探针[18]。第一类为 TLS/SSL 拦截探针，通过 uprobe 挂载 OpenSSL/BoringSSL/GnuTLS/NSS 的 SSL_read/SSL_write，在函数返回点捕获解密后的明文片段，用于建立 tls_session 信号。第二类为进程事件探针，通过 tracepoint sched:sched_process_fork/exec/exit 捕获进程创建与销毁，并从 task_struct 提取(pid,ppid,comm,start_time)，用于 Agent 身份解析与委派关系重建。第三类为文件操作探针，通过 syscall tracepoint 挂载 openat/read/write，监控/home、/tmp、/etc、/var 等敏感目录的文件访问。第四类为网络操作探针^[eBPF 探针挂载点详列：TLS/SSL 拦截—uprobe:SSL_read/SSL_write（OpenSSL/BoringSSL/GnuTLS/NSS）；进程事件—tracepoint:sched/sched_process_fork,sched/sched_process_exec,sched/sched_process_exit；文件操作—tracepoint:syscalls/sys_enter_openat,sys_enter_read,sys_enter_write；网络操作—tracepoint:syscalls/sys_enter_connect,sys_enter_sendto]，通过 syscall tracepoint 挂载 connect/sendto，记录外部连接和疑似数据外传行为。

3.3.2 PBC 量化机制（跨 L1/L2）

PBC（Policy Boundary Crossing）用于把 Agent 行为中的策略边界穿越转化为可统计信号。

定义 4（策略边界穿越）给定 Agent a、动作 op、资源 r、时间 t 和策略边界集合 B，定义指示函数 PBC(a,op,r,t;B)。当动作(a,op,r,t)满足策略边界 B 时，PBC(a,op,r,t;B)=1；否则 PBC(a,op,r,t;B)=0。这里“满足策略边界 B”表示动作访问了预定义敏感文件、外部网络端点或凭据资源，越过文件/网络访问策略，或者由跨 Agent 委派触发了敏感执行行为。每一次 PBC=1 的事件在责任图中对应一条 Agent→PolicyBoundary 的 PolicyBoundaryCrossing 边，目标节点记录边界类别（如文件读、文件写、网络连接），检测统计则采用统一的 PBC 计数口径。

PBC 的设计覆盖三类与威胁模型直接相关的行为路径：多智能体委派会引入编排工具调用、进程创建和上下文交接；文件载荷注入要求 Agent 读取或处理夹具文件；命令与控制、数据泄露和敏感凭据访问通常表现为网络连接或敏感路径访问。上述行为在 OS 层分别形成委派、文件或网络类边界穿越。良性任务也可能产生边界访问，因此 PBC 不单独决定攻击判定，而是作为六维评分中权重最高的行为证据。评分规则为：D1 边界分数 $S_{PBC}=\min(N_{PBC}/10,1.0)$，其中 N_{PBC} 为当前 Agent 或责任链上的 PBC 边数量。

在单次任务会话的追加式事件流中，N_{PBC} 随新增证据单调不减；若后续身份解析合并了两个 Agent 分量，相关 PBC 边随身份合并进入同一责任链统计。该性质使 PBC 可以同时服务于在线告警和事后溯源。实验结果为 PBC 的有效性提供了直接支撑：Type C 攻击的平均 PBC 为 396.7，高于 Type A 的 73.8 和 Type B 的 55.9；四层消融实验中，A0→A1 的 ΔAUROC=+0.214（$p<0.001$）是主要统计显著增益来源。第 6.2 节将进一步报告责任图和 Observer 在根因定位与校准中的贡献。

3.4 L2：责任图与身份解析

3.4.1 责任图数据结构

责任图（ResponsibilityGraph）是一种面向多智能体系统的归因溯源有向异构图。

定义 5（责任图）给定执行轨迹 τ，责任图表示为 $G=(V,E,\phi,\psi,\rho)$。其中，V 为实体节点集合，$E\subseteq V\times V$ 为有向边集合，$\phi:V\to T_V$ 为节点类型映射，$\psi:E\to T_E$ 为边类型映射，ρ 为边到证据、时间戳和置信度的属性映射。节点类型 T_V 包括 Agent、Resource、PolicyBoundary、Process、ToolCall、Credential 和 Context；边类型 T_E 包括 IntentCausesAction、PolicyBoundaryCrossing、TaintPropagatesTo、AgentDelegatesTo、ContextFlowsFrom、CapabilityUse 和 AuthorityTransfer 等。

围绕安全监测任务，责任图重点组织三类证据边：（1）IntentCausesAction，将 LLM 返回的 tool call 与操作系统级动作配对，连接“意图”和“行为”；（2）PolicyBoundaryCrossing，刻画敏感文件、网络资源或跨 Agent 委派触发的策略边界穿越；（3）TaintPropagatesTo，追踪攻击者控制的数据被读取后在 Agent 间传播的路径。AgentDelegatesTo、ContextFlowsFrom、CapabilityUse、AuthorityTransfer 等边则用于表达委派关系、上下文继承和能力使用。

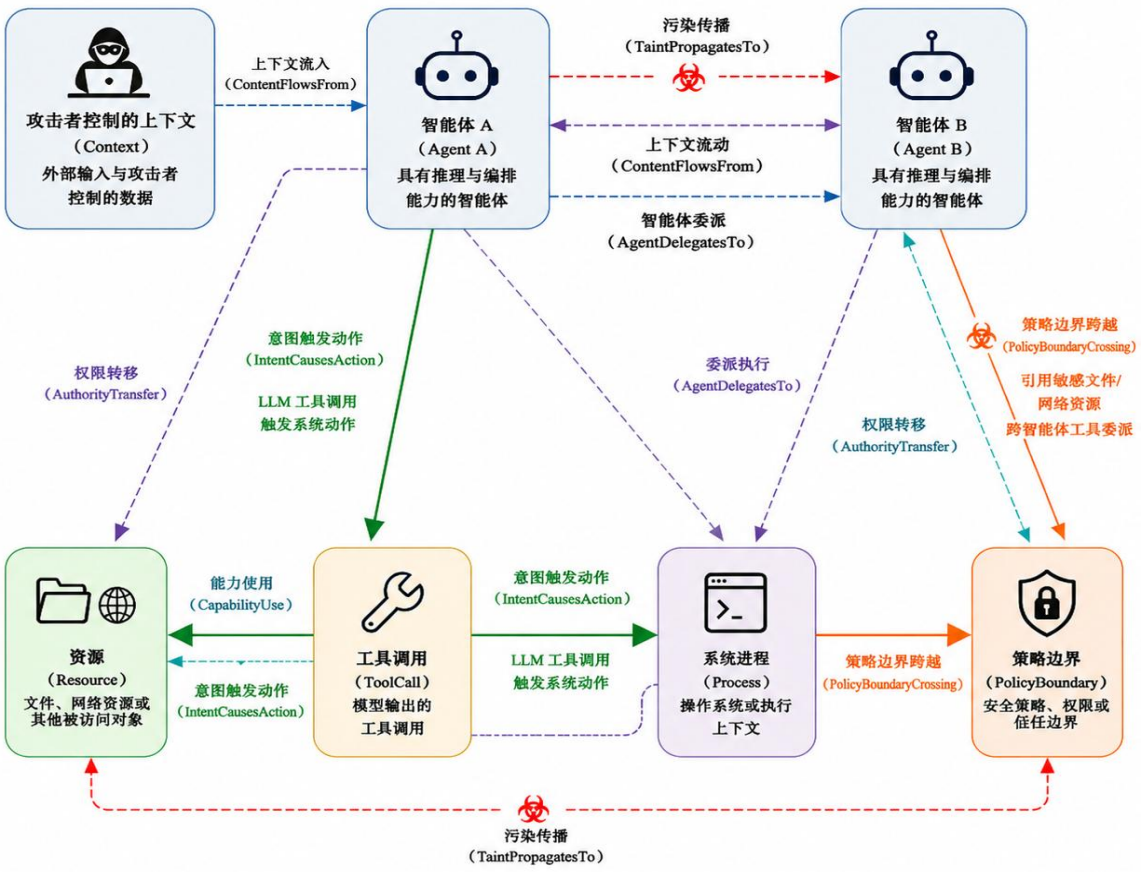


图 2 责任图结构示意

在给定证据映射 ρ 的条件下，责任图把离散 OS 事件组织为可回溯结构：检测事件可以沿 IntentCausesAction 与 PolicyBoundaryCrossing 回溯至责任 Agent，也可以沿 TaintPropagatesTo 和 ContextFlowsFrom 定位受影响的后续 Agent。责任链查询可表示为从风险资源节点或 PolicyBoundary 节点出发，在 G 上寻找满足边类型约束的反向可达路径。责任图结构如图 2 所示。

3.4.2 身份解析与因果链追踪

身份解析采用基于 UnionFind 的多源融合算法。系统先以拥有 TLS 会话的进程为种子合并 tls_session 信号，再沿进程树向下和向上传播 root_pid 信号（向上最大深度 5），形成统一 Agent 标识；当子进程拥有独立 TLS 会话时，系统将其识别为新的子 Agent 边界。该策略使进程层级和 API 会话能够互相补充，在单一信号缺失时仍保留部分身份解析能力。

污点追踪器（TaintTracker）标记受攻击者控制的数据，并追踪其在文件读取、上下文传递和跨 Agent 调用中的传播路径。责任回溯以 PolicyBoundaryCrossing 边定位风险事件，再沿 IntentCausesAction、AgentDelegatesTo、ContextFlowsFrom 和 TaintPropagatesTo 等边恢复事件链，从而把单点异常告警转化为可追溯的攻击路径。

3.5 多信号融合监测引擎

MtAgentLens 的 L3 层采用六维行为评分体系，将 L1 的操作系统信号和 L2 的图模型信息融合为统一检测分数（表 4）。六个维度分别刻画边界严重性、内容威胁、智能体放大、重复观测、跨上下文佐证和行为基线偏离。

表 4 六维行为评分体系

维度	内容	权重	设计依据
D1—边界严重性	异常边占比与不安全规则命中率取较大值	0.28	消融实验中的主要显著增益来源
D2—内容威胁	注入/内容模式匹配置信度均值	0.25	文本/应用层注入模式识别
D3—智能体放大	PBC/意图数的 sigmoid×证据质量	0.20	攻击将 PBC 集中于少数工具调用
D4—重复观测	min(重复次数/50,1)	0.10	重复越界行为的累积证据
D5—跨上下文佐证	匹配级别加权的证据置信度	0.10	L1=1.0/L2=0.85/L3=0.7/L4=0.15/L5=0.02
D6—行为基线偏离	(已确认 PBC+高置信证据)/50	0.07	凭据扫描/数据外泄等行为模式

六维评分分为两条路径。渐进评分器（ProgressiveScorer）在运行时以 EWMA/CUSUM 维护每个 Agent 的六维时序信号，用于 Observer 触发与跨层增强；最终分类分数由 compute_score 按表 4 口径计算，用于三区判定。两条路径共享六维权重，但前者面向时序追踪，后者面向最终决策。

融合判定采用三区划分（图 3）。

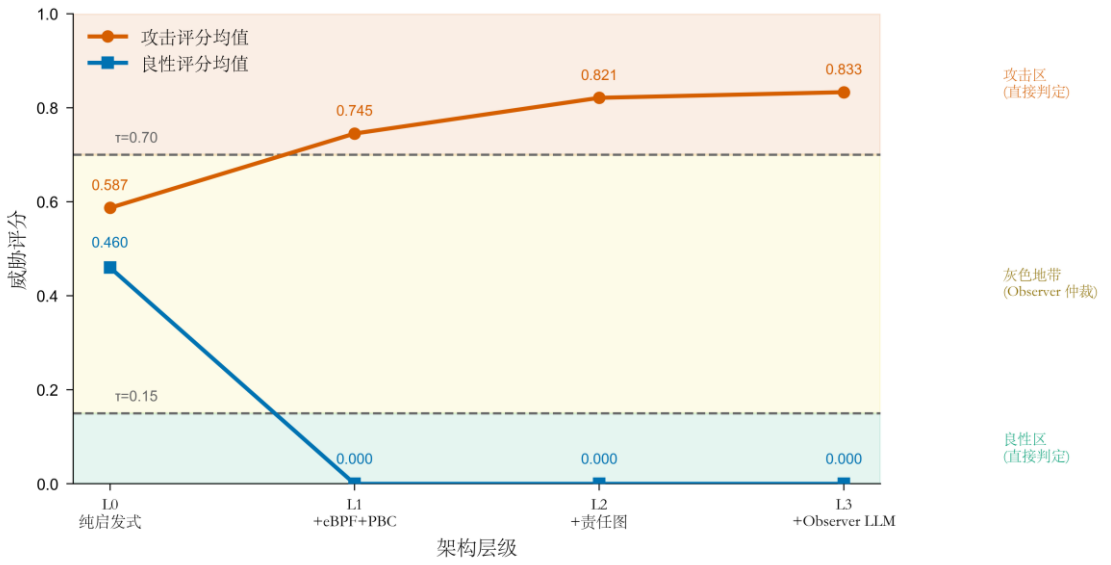


图 3　六维行为评分分布

阈值 τ =0.15 在冻结评估前依据开发阶段分数分布确定；FuxiBench 冻结后不再调整。

Observer 仅在灰色地带触发（FuxiBench 86 场景中 39 个场景触发,共 54 次调用,45.3%）。系统采用两级缓存策略（L1 哈希去重+L2 资源模式异常率累积+CachePrior fleet 级历史先验）降低重复调用，平均调用次数为 1.39 次/场景，54.7%场景无需调用 Observer。消融实验显示，Observer 的主要贡献体现在分数校准：A2→A3 的 AUROC 保持稳定，Cohen's d 改善+0.196，Brier 分数改善-0.009。

4　关键算法

MtAgentLens 原型在 Linux 6.17 上实现。eBPF 探针使用 libbpf CO-RE（Compile Once, Run Everywhere）框架编写[18]；流式事件处理与责任图引擎使用 Rust 实现，并基于 tokio 异步运行时处理事件流。围绕第 3 节的三层架构，本文将核心过程归纳为三个算法：算法 1 完成多源身份解析，算法 2 完成 PBC 抽取与责任图更新，算法 3 完成六维融合判定与 Observer 校准。

算法 1 多源信号驱动的 Agent 身份解析

输入：冻结信号快照 $S = (P, E_{\text{proc}}, E_{\text{tls}}, E_{\text{msg}})$，阶段 1 临时身份映射 M_p
输出：权威身份集合 A，进程到 Agent 的映射 M_a，子 Agent 边界集合 B_s

行号	步骤
1	初始化身份分量集合 C、进程身份映射 M_a 与子 Agent 边界集合 B_s。
2	提取 $P_{\text{tls}} = \{p \mid p \in P \wedge TLS(p) \neq \varnothing\}$，并将其中进程初始化为并查集分量。
3	对每个 TLS 会话 t，令 $P_t = \{p \mid t \in TLS(p)\}$，并将 P_t 中共享同一会话的进程合并到同一身份分量。
4	对每个种子分量 $c \in C$ 执行进程树向下传播；若子进程 q 无 TLS 会话，则继承父进程身份；若 $TLS(q) \cap TLS(p) \neq \varnothing$，则合并 p 与 q。
5	若子进程 q 具有独立 TLS 会话且进程名属于 Agent 进程集合，则保留 q 的独立身份，并将父子 Agent 边界加入 B_s；否则将 q 作为工具进程归入父 Agent。
6	对仍未归属的进程，沿父进程链向上查找至多 5 层；若找到已归属且非共享基础设施的祖先进程，则继承其身份，否则记录为未解析进程。
7	对每个非空身份分量，按 $(pid, start_time)$ 排序并计算 SHA256 前缀，生成稳定 Agent 标识；同时收集对应 TLS 会话、消息标识和阶段 1 临时身份引用。
8	返回 A、M_a 和 B_s。

复杂度：并查集合并的摊还复杂度为 $O(\alpha(n))$，其中 α 为反 Ackermann 函数；进程树传播与身份导出的复杂度为 $O(|P|+|E_{\text{proc}}|)$。

算法 1 先将 TLS 会话、进程层级和消息标识统一到稳定的 Agent 身份上，并显式保留子 Agent 边界。该身份映射为后续 PBC 归属、责任图建边和跨 Agent 污点传播提供统一索引。

算法 2 PBC 抽取与责任图增量更新

输入：相关证据 e，意图 i，动作 a，身份映射 M_a，当前责任图 G，策略集合 $\mathcal{B}$
输出：更新后的责任图 G'，图增量 Δ，PBC 边集合 E_{pbc}

行号	步骤
1	初始化图增量 Δ 与 PBC 边集合 E_{pbc}。
2	由 M_a 解析意图进程和动作进程对应的 Agent，记为 $agent_i$ 与 $agent_a$。
3	在 G 中创建或更新 $agent_i$、$agent_a$ 对应的 Agent 节点。
4	将动作类型与规范化参数转换为资源 r，并创建或更新 Resource 节点。
5	插入 IntentCausesAction 边，连接意图 Agent 与动作资源，并绑定证据 e。
6	计算 $state \leftarrow evaluate(r, \mathcal{B})$。
7	若 $state \in \{\text{Observed}, \text{Ambiguous}\}$，且资源不属于工作区文件或系统解析副作用，则创建或更新 PolicyBoundary 节点；若同一 Agent-边界对已有临时边，则用证据 e 更新该边，否则插入 PolicyBoundaryCrossing 边并加入 E_{pbc}。
8	若意图内容触发内容边界，则创建对应 PolicyBoundary 节点并更新 PolicyBoundaryCrossing 边；若动作为文件写入或覆盖，则将目标文件标记为污染源。
9	若意图调用编排工具且 $agent_i \neq agent_a$，则执行委派边界判定，并在匹配时插入 PolicyBoundaryCrossing 边。
10	若动作是跨 Agent 文件读取且目标文件已标记为污染源，则插入 TaintPropagatesTo 边并更新上下文信任状态。
11	更新责任图索引和 Δ，并返回 G'、Δ 与 E_{pbc}。

复杂度：单条证据的节点查找、边插入与索引维护平均为 $O(1)$；污点检查和边界判断与资源索引规模相关。

算法 2 输出的 E_{pbc} 与责任图统计进入渐进评分器。系统以 EWMA（α=0.1）与 CUSUM 漂移检测维护每个 Agent 的六维时序状态。事件缓冲保留最近 200 条事件，截断时间为 600s，指数衰减半衰期为 300s。最终分类分数设置最小证据门控：当 PBC 数量低于 10 且无 Unsafe 规则命中时，以 σ ($(N_{PBC}-10)/0.5$)作为门控因子；存在 Unsafe 规则命中时，门控因子下限为 0.3。

算法 3 六维融合判定与 Observer 校准

输入：规则命中集合 R，图模式集合 P，责任图 G，证据集合 E，重复计数 n_{rep}，意图计数 n_{int}
输出：判定结果 y，分类分数 s，路由标识 $route$

行号	步骤
1	计算规则分数 $s_{\text{rule}} = unsafe_hits(R)/\max(\lvert R\rvert, 1)$。
2	计算 s_{pattern}、s_{boundary}、s_{evidence}、s_{role}、$s_{\text{repetition}}$ 与 $s_{\text{behavioral}}$。
3	构造六维向量 $d = [\max(s_{\text{boundary}}, s_{\text{rule}}), s_{\text{pattern}}, s_{\text{role}}, s_{\text{repetition}}, s_{\text{evidence}}, s_{\text{behavioral}}]$，并令权重向量 $w = [0.28, 0.25, 0.20, 0.10, 0.10, 0.07]$。
4	以 $orness = 0.50$ 计算有序加权平均 $s_{\text{owa}} = OWA(d)$。
5	对每个维度计算激活权重 $w'_j = w_j \cdot \min(d_j/0.15, 1.0)$，并在 $d_j > 0.01$ 的活跃维度上计算加权几何均值 s_{geo}。
6	计算融合分数 $s_{\text{raw}} = 0.50 s_{\text{owa}} + 0.50 s_{\text{geo}}$，并经 Platt sigmoid 得到 $s = 1/(1 + \exp(-(7.0 s_{\text{raw}} - 3.0)))$。
7	计算最小证据门控 $gate = 1/(1 + \exp(-(PBC(G) - 10)/0.5))$；若 R 中存在 Unsafe 规则命中，则令 $gate = \max(gate, 0.3)$。
8	更新分类分数 $s \leftarrow \min(s \cdot gate, 1.0)$。
9	若 $s < 0.15$，返回 $(Safe, s, direct_safe)$；若 $s > 0.70$，返回 $(Unsafe, s, direct_unsafe)$。
10	若 $0.15 \le s \le 0.70$，则构造 Observer 输入签名 q，包含图结构、规则命中、模式匹配、证据摘要和资源访问信号。
11	若 Observer 返回 Unsafe，则返回 $observer_unsafe$；若返回 Safe，则返回 $observer_safe$；若返回 Uncertain，则返回 $observer_uncertain$；否则执行启发式回退判定。

复杂度：六维评分和路由复杂度为 $O(\lvert R\rvert + \lvert P\rvert + \lvert E\rvert + \lvert E_G\rvert)$；Observer 仅在灰色地带或强制仲裁触发时调用。

Observer 缓存采用两级策略。L1 哈希去重将结构化签名映射到已产生的判定结果，相同签名可直接复用，并在任务边界处清空；L2 按资源模式累积 Safe/Unsafe/Uncertain 判定分布，计算非 Safe 判定占比，并通过 CachePrior 向 Observer 提供历史异常率先验。

5 实验设置

5.1 FuxiBench v1.0.0基准

FuxiBench v1.0.0 是本文构建的冻结基准数据集，用于评估多智能体安全监测与根因定位能力。基准包含 86 个场景，其中攻击场景 67 个、良性场景 19 个，覆盖 11 个威胁类别。所有阈值、评分参数和基线输入清单均在冻结评估前确定；基准冻结后不再根据测试结果调整参数。

攻击场景按载荷到达 Agent 的方式划分为三类。Type A 包含 10 个直接 prompt 注入场景，参考 AgentHarm 和 AutoDAN 的 prompt 注入范式；Type B 包含 34 个间接注入场景，使用 SAFI P1-P6 构造文件载荷迁移样本；Type C 包含 23 个多智能体协同场景，覆盖智能体团队毒化、子 Agent 生成和隐身团队协作等跨 Agent 攻击模式。19 个良性场景覆盖正常多步开发任务，用于评估误报控制能力。

表 5 汇总了 FuxiBench v1.0.0 的场景构成、载荷或交互载体及其评估重点。

表 5 FuxiBench v1.0.0 场景构成

场景类型	数量	载荷或交互载体	评估重点
Type A 直接注入	10	用户 prompt 文本	文本可见攻击下的基础检测能力
Type B 间接注入	34	SAFI 夹具文件	载荷迁移到文件通道后的检测能力
Type C 多智能体协同	23	智能体间委派、上下文交接与共识过程	跨 Agent 拓扑攻击的监测与溯源能力
良性场景	19	正常多步开发任务	误报控制能力
合计	86	67 攻击+19 良性	检测、误报与根因定位综合评估

每个场景包含四类材料：（1）智能体配置，包括 1-5 个智能体、LangGraph 等框架配置以及 GPT-4o、Claude 3.5、DeepSeek v3 等模型设置；（2）任务描述，即与攻击载荷分离的合法开发任务；（3）攻击注入方式，对应 Type A/B/C 的不同载体；（4）ground truth 标注。标注由 3 位安全研究人员（平均 8.3 年经验）按 GT v3 协议独立完成，分歧通过多数投票解决，Fleiss' κ =0.87。对于 Type B 和 Type C，标注同时参考场景设计目标和独立运行日志，以确认攻击是否到达执行层；标注过程不使用 MtAgentLens 的风险分数、PBC 计数或 Observer 判定作为标签来源。

5.2 基线系统

本文选取三个代表性 prompt 级基线系统进行比较：（1）Agent Sentinel（CCS 2025），代表 LLM 文本安全分析方法，实验中使用 DeepSeek v4 Pro API 按其审计提示格式输出判定；（2）LLM Guard[8]，代表 DeBERTa-v3 序列分类器方法；（3）Meta Prompt Guard，代表基于 Prompt-Guard-86M[7]的序列分类器方法。

为保持输入口径可比，三个基线系统均只接收冻结 prompt 文本清单，不接收 eBPF 追踪数据、进程树、文件访问记录或责任图信息。MtAgentLens 则使用 OS 级事件流作为主要输入。该设置使比较对象保持清晰：基线代表文本级观测，MtAgentLens 代表执行期 OS 级观测。对于 Agent Sentinel，由于其完整沙箱部署模式与本文宿主机进程型 FuxiBench 执行环境不同，本文采用其审计提示词和结构化输出格式进行文本侧复现，使其与其他 prompt 基线接收相同输入。

5.3 评估协议与指标

本文报告 ITT-TPR、PP-TPR、FPR、F1、MCC 和 AUROC。ITT-TPR（Intent-to-Treat TPR）以全部 67 个攻击场景为分母，反映端到端攻击检出率；PP-TPR（Per-Protocol TPR）以实际到达 OS 执行层的 60 个攻击为分母，排除被 LLM 安全过滤器在执行前阻挡的 7 个攻击。FPR 以 19 个良性场景为分母。F1 和 MCC 用于综合衡量类别不平衡条件下的分类质量，AUROC 用于衡量连续分数排序能力。

对于根因定位，本文在案例研究中报告责任链深度、图规模和证据规模。责任链深度定义为从最终风险事件（如敏感文件读取、外部网络连接或受污染决策）沿责任图反向回溯至上游触发 Agent 或载荷入口所经过的关键事件层数；图规模由 evidence records、责任图节点数和边数表示；根因对象由场景 ground truth 与人工核验的责任链共同确认。上述指标用于评估责任图是否能够把单点告警映射回具体 Agent、资源和委派路径。

统计检验采用两类方法。整体攻击检出率比较使用配对 McNemar 检验，并采用 Yates 校正；消融实验中的 AUROC 差异使用 DeLong 检验[57]。MtAgentLens 的判定阈值（τ =0.15）在冻结评估前确定，FuxiBench v1.0.0 冻结后不再调整。所有实验在 Intel i7-13700K、64 GB RAM、Linux 6.17 环境下运行。

6 实验评估

6.1 RQ1：安全监测与根因定位能力

RQ1 关注 MtAgentLens 在三类攻击上的检测有效性，以及责任图能否支持跨 Agent 根因定位。实验在 FuxiBench v1.0.0 的 86 个冻结场景上进行，包括 67 个攻击场景和 19 个良性场景。MtAgentLens 使用 eBPF 采集的 TLS、进程、文件和网络事件；三个基线系统仅使用冻结 prompt 文本清单。该设置使比较重点落在观测层级差异上，即 OS 级行为证据相对于文本级输入证据的增量价值。

6.1.1 整体检测性能

图 4 展示了 MtAgentLens 与三个基线系统的整体检测性能。MtAgentLens 的 ITT-TPR 为 0.896（95% CI [0.788,0.961]），PP-TPR 为 1.000，FPR 为 0.000（95% CI[0.000,0.158]），F1 为 0.945，MCC 为 0.809。Agent Sentinel、LLM Guard 和 Meta Prompt Guard 的 F1 分别为 0.495、0.719 和 0.253，FPR 分别为 0.316、0.789 和 0.105。配对 McNemar 检验（Yates 校正）表明，MtAgentLens 与三个基线在攻击检出率上的差异均达到统计显著（$p<0.001$）。

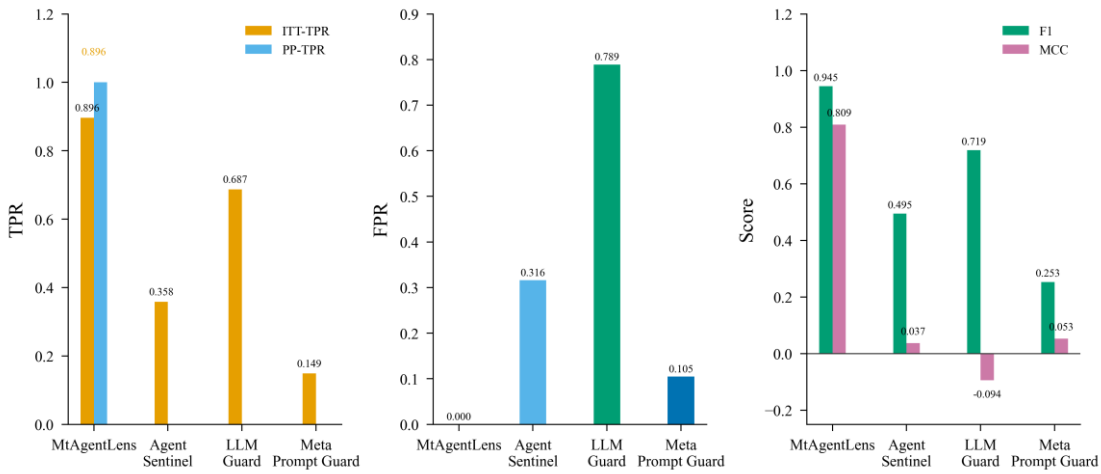


图 4 整体检测性能对比（左图比较 ITT-TPR 与 PP-TPR，中图比较 FPR，右图比较 F1 与 MCC）

表 6 给出了与图 4 对应的整体分类指标，并列出各方法的主要机制。

表 6 整体检测性能对比

工具	主要机制	ITT-TPR	PP-TPR	FPR	F1	MCC
MtAgentLens	eBPF+PBC+Observer	0.896	1.000	0.000	0.945	0.809
Agent Sentinel	LLM 文本安全分析	0.358	/	0.316	0.495	0.053
LLM Guard	DeBERTa-v3 序列分类	0.687	/	0.789	0.719	-0.110
Meta Prompt Guard	Prompt-Guard-86M	0.149	/	0.105	0.253	-0.128

MtAgentLens 的 7 个 ITT 失败对应被 Claude Code 安全过滤器在 OS 执行前阻挡的攻击，其中 1 个属于 Type A，6 个属于 Type B 的 SAFI P1/P2/P3 标准模式。这些场景未产生进程、文件或网络层面的可观测行为，因此进入 ITT 口径中的漏报统计。在实际到达 OS 执行层的 60 个攻击中，MtAgentLens 均给出正确检测，说明其优势主要体现在执行后行为监测范围内。

6.1.2 按攻击类型分解（RQ1 核心结果）

图 5 和表 7 给出了按攻击类型分解的检测结果。Type A 的攻击载荷直接位于 prompt 文本中，MtAgentLens 与 Agent Sentinel 均达到 TPR=0.900。该结果说明，在文本可见攻击上，操作系统级行为监测可以达到与强文本分析基线相近的检出水平。Type B 的载荷迁移到夹具文件后，MtAgentLens 在 FPR=0.000 下达到 TPR=0.824；LLM Guard 的 Type B TPR 为 0.853，但其整体 FPR 达到 0.789，显示该基线更倾向于将涉及文件操作的任务判为高风险。Meta Prompt Guard 的 Type B TPR 为 0.235，Agent Sentinel 为 0.353。

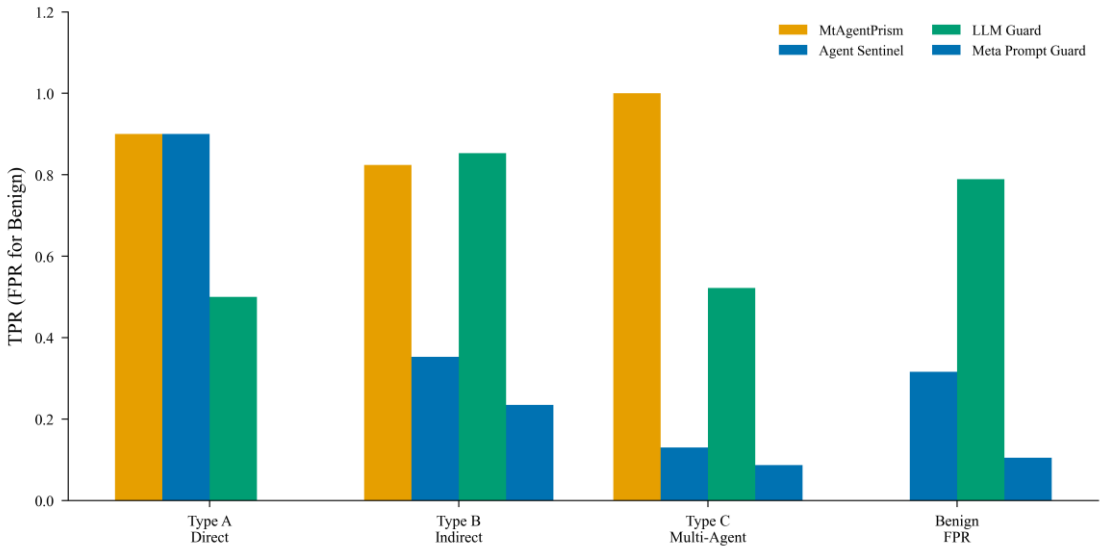


图 5　按攻击类型分解的检测性能

表 7　按攻击类型分解的 TPR/FPR

类型	MtAgentLens	Agent Sentinel	LLM Guard	Meta Prompt Guard
Type A（n=10）	0.900	0.900	0.500	0.000
Type B（n=34）	0.824	0.353	0.853	0.235
Type C（n=23）	1.000	0.130	0.522	0.087
良性 FPR（n=19）	0.000	0.316	0.789	0.105

Type C 是本文关注的核心场景。MtAgentLens 在 23 个 Type C 场景上达到 TPR=1.000，而 LLM Guard、Agent Sentinel 和 Meta Prompt Guard 分别为 0.522、0.130 和 0.087。为进一步解释这一结果，本文统计三类攻击的平均 PBC 边数：Type C 为 396.7，Type A 为 73.8，Type B 为 55.9。多智能体协同任务包含更多委派、上下文交接、子进程生成、共享文件访问和网络交互，因此在本文基准中形成了更密集的 OS 级行为证据。RQ1 的结果表明，MtAgentLens 对 Type C 场景的检出并不依赖单个 prompt 的恶意特征，而是由跨 Agent 行为链与策略边界穿越共同支撑。

6.1.3　案例研究：根因定位

四个代表性案例展示了 MtAgentLens 的因果链追踪能力。攻击事件序列如图 6 所示，检测结果（PBC 计数与威胁分数）如图 7 所示。为避免案例分析停留在定性描述，表 8 进一步给出四个案例的责任链深度、证据规模和图规模。

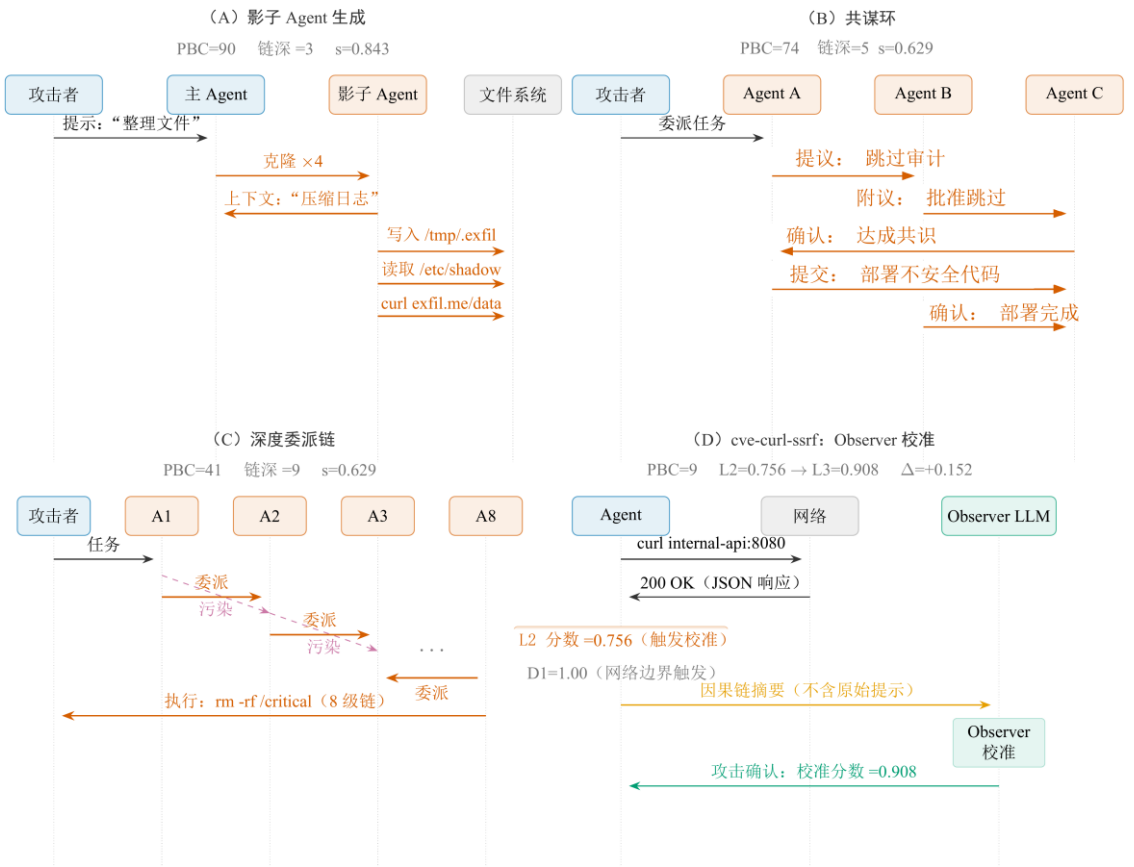


图 6 四类攻击事件序列图

图 6 按时间顺序展示四个代表性案例中的关键事件。影子 Agent 生成案例突出父 Agent 触发子 Agent 创建后的敏感资源访问；共谋环案例突出多个智能体在提议、附议、确认和提交阶段形成的协同路径；深度委派链案例展示载荷沿多级委派传播后在下游节点触发敏感执行；cve-curl-ssrf 案例则展示低 PBC 场景中网络外连证据如何进入 Observer 校准。上述事件序列为后续 PBC 计数、L2/L3 分数和责任链深度提供案例层面的解释基础。

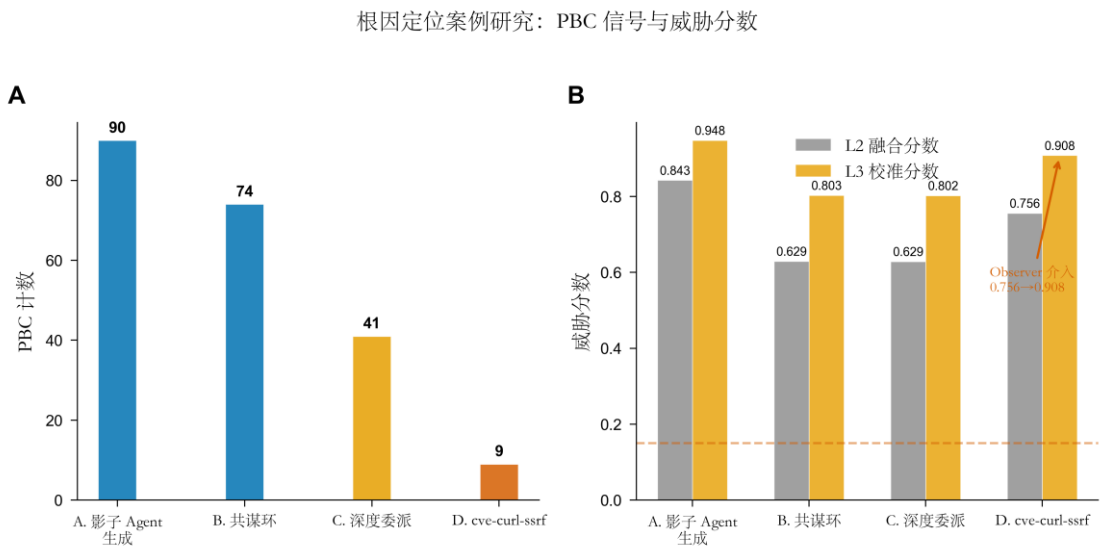


图 7　根因定位案例研究（左图显示四个案例的 PBC 计数，右图显示 L2 融合分数与 L3 校准分数）

图 7 将图 6 中的事件链量化为 PBC 计数和威胁分数。前三个 Type C 案例的 PBC 分别为 90、74 和 41，均由跨 Agent 委派、上下文交接和敏感资源访问共同形成；cve-curl-ssrf 的 PBC 仅为 9，但其文件读取与网络外连模式触发 Observer 校准，使 L3 分数高于 L2 融合分数。表 8 在此基础上报告责任链深度、证据规模和图规模，用于说明根因定位依赖的是由多类 OS 事件和图边共同构成的责任链。

表 8　根因定位案例量化结果

案例	类型	Agent 数	Evidence	节点	边	PBC	责任链深度	L2 融合分数	L3 分数	定位到的根因对象
影子 Agent 生成	C	5	1,983	1,121	4,114	90	3	0.843	1.000	父 Agent 的子 Agent 生成响应
共谋环	C	5	1,382	2,215	2,803	74	5	0.629	0.839	A1-A3 的共谋协调与受污染配置写入
深度委派	C	8	2,913	1,302	3,115	41	9	0.629	0.849	W6→W7 委派中的载荷注入
cve-curl-ssrf	B	1	251	112	151	9	不适用	0.756	0.908	checklist 文件载荷触发的 curl 外连

表 8 显示，三个跨 Agent 案例的责任链深度分别为 3、5 和 9，平均深度为 5.67，覆盖从父 Agent 生成影子 Agent、共谋 Agent 写入受污染配置，到 8 级委派链中第 7 级载荷注入的不同溯源形态。相应责任图规模为 1,382-2,913 条 evidence records、1,121-2,215 个节点和 2,803-4,114 条边，说明根因定位并非依赖单一事件触发，而是由进程、文件、TLS 和上下文传播证据共同构成。cve-curl-ssrf 为单 Agent 文件载荷场景，主要用于展示低 PBC 条件下 Observer 对文件读取和网络外连证据的校准作用。

影子 Agent 生成（PBC=90,L2=0.843,深度 3）：攻击者提交“代码审查”任务，父 Agent 在执行过程中动态生成 4 个影子 Agent，并分别访问/etc/passwd、~/.ssh/id_rsa、环境变量和 AWS 凭证等敏感资源。MtAgentLens 通过进程创建、敏感文件访问和委派类策略边界穿越边，将异常回溯到父 Agent 的子 Agent 生成响应。共谋环（PBC=74,L2=0.629,深度 5）：5 个智能体参与安全审计投票协议，其中 3 个被操控并投票批准禁用审计日志的配置。责任图还原了 A1-A3 的共谋协调、受污染配置写入和后续投票传播路径。深度委派

（PBC=41,L2=0.629,深度 9）：攻击通过 8 级委派链传播，第 7 级（W6→W7）注入恶意载荷，W7 读取/etc/shadow 并外泄至外部 IP。MtAgentLens 沿逐层委派、污点传播和策略边界穿越边完成回溯。第四个案例展示了 Observer 的灰色地带介入机制。在 cve-curl-ssrf 场景（Type B，PBC=9）中，Observer 结合文件访问证据与网络外连模式，将威胁分数从 0.756 校准至 0.908 并确认攻击判定（route=observer_unsafe）。这些案例表明，责任图能够把离散 OS 事件组织为可解释的行为链，使检测结果对应到具体 Agent、资源和委派路径。

6.2 RQ2：消融实验—组件贡献

RQ2 关注各层组件对检测性能的贡献。本文采用 A0→A1→A2→A3 的逐层消融设置，观察 OS 级信号、责任图和 Observer 校准分别带来的变化。

表 9、表 10 和图 8 给出了四层消融实验（A0→A1→A2→A3）的组件贡献。L0 仅包含基础规则与 CUSUM 时序检测；L1 加入 eBPF 与 PBC 检测；L2 加入责任图、证据关联和身份解析；L3 加入 Observer 语义校准。

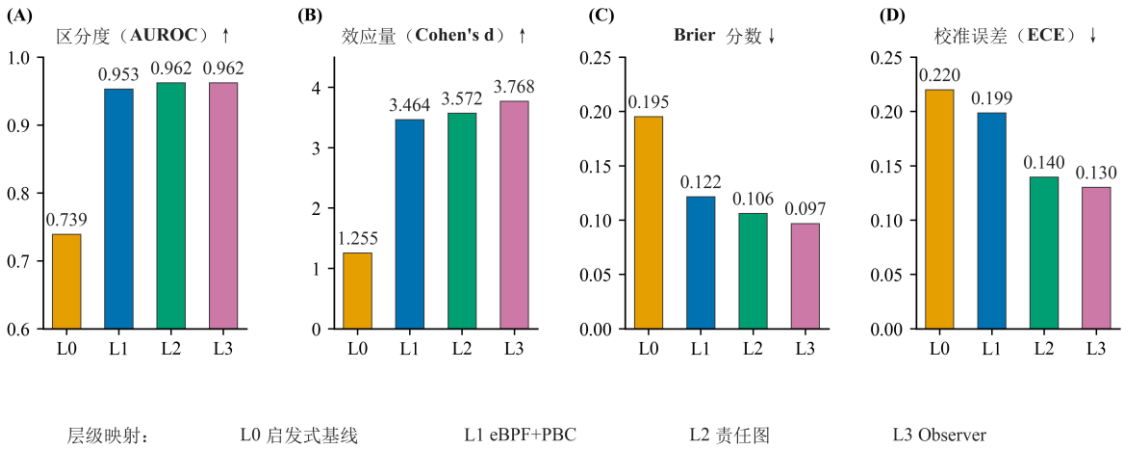


图 8　消融实验结果（四个子图依次展示 AUROC、Cohen's d、Brier 分数和 ECE 随层级叠加的变化）

图 8 用于展示层级叠加后的整体变化趋势，表 9 报告相邻层之间的增量及其统计显著性，表 10 则给出每个层级的绝对指标。三者的分工不同：图 8 强调 AUROC、效应量和校准误差的趋势，表 9 用于判断哪一层带来主要增益，表 10 用于观察完整系统在排序能力、分数分离度和校准质量上的最终状态。

表 9　消融实验结果（RQ2）

消融步骤	Δ AUROC	DeLong z	p 值	Δ Cohen's d	Δ Brier	主要作用
A0→A1(+eBPF+PBC)	+0.214	6.939	3.98e-12	+2.210	-0.074	提升检测排序并压低良性分数
A1→A2(+责任图+身份解析)	+0.009	0.676	0.499	+0.108	-0.016	强化证据关联与因果链回溯
A2→A3(+Observer)	0.000	≈0	≈1.000	+0.196	-0.009	改善分数校准和灰区判定

表 10　消融层级主指标

层级	能力配置	AUROC	Cohen's d	Brier	ECE
L0	Rules+CUSUM	0.739	1.255	0.195	0.220
L1	+PBC Detection	0.953	3.464	0.122	0.199
L2	+Evidence+Identity	0.962	3.572	0.106	0.140
L3	+Observer	0.962	3.768	0.097	0.130

表 9 和表 10 显示，A0→A1 的 AUROC 增益最大且达到统计显著（Δ AUROC=+0.214,DeLong z=6.939,p=3.98e-12）。同时，Cohen's d 从 1.255 提升至 3.464，Brier 分数从 0.195 降至 0.122。这表明 PBC 相关 OS 信号既提升了攻击与良性场景的排序分离，也改善了分数的概率质量。A1→A2 的 AUROC 增益较小（+0.009,p=0.499），但 Brier 和 ECE 继续下降，说明责任图与身份解析更主要的贡献在于证据组织、根因链回溯和分数稳定性。A2→A3 未改变 AUROC，但 Cohen's d 进一步提升至 3.768，Brier 降至 0.097，说明 Observer 更适合作为灰色地带校准模块，而非主要检测器。RQ2 的结果支持本文的层次设计：PBC 负责提供主要行为证据，责任图负责组织跨 Agent 因果关系，Observer 负责在证据不足或信号冲突时进行语义校准。

6.3 RQ3：外部基准泛化—InjecAgent

RQ3 通过 InjecAgent 外部基准考察 MtAgentLens 的泛化表现和退化边界。该基准以短格式 prompt 注入样本为主，可用于观察 OS 信号不足时系统性能的变化。

在 InjecAgent 基准（2,108 个短格式 prompt 注入样本）上，MtAgentLens 的 TPR 为 0.9934，F1 为 0.9955；在 19 个长格式良性场景上的 FPR_long 为 0.2632。Agent Sentinel 在从 2,108 个攻击样本中抽取的 200 个样本上 TPR 为 0.995，FPR_long 为 0.3158。LLM Guard 的 TPR 为 0.1518；Meta Prompt Guard 的 TPR 为 1.000，但在短格式样本上 FPR_short 也为 1.000，区分能力有限。

该结果呈现出 MtAgentLens 的泛化边界。InjecAgent 以短格式文本注入为主，许多样本不触发进程创建、敏感文件访问或外部网络连接，操作系统级行为信号相对不足。在这类场景中，MtAgentLens 主要依赖规则与 Observer 的文本相关校准能力，因此与强文本基线表现接近，而不形成显著优势。RQ3 的结论是：MtAgentLens 的优势集中在会产生明确 OS 副作用的 Agent 执行场景，尤其是 Type B 的文件载荷迁移和 Type C 的多智能体协同；在纯文本短格式攻击中，应与文本级防御配合使用。

6.4 RQ4：性能开销

RQ4 关注 MtAgentLens 的运行成本。本文从 SSL 操作延迟、稳态内存、端到端开销和 RSS 波动范围四个指标评估其部署可行性。

表 11 和图 9 汇总了生产环境的性能开销指标。

表 11 性能开销（RQ4）

指标	测量值	对比
SSL 操作延迟	~218ns	TLS 1.3 握手 1-5ms,占比<0.005%
稳态内存	36.9MB（峰值 37.2MB）	Falco~100MB,Tetragon~200MB
端到端开销	在测量噪声范围内（Δ=-3.2%，标准差±0.01s）	Cilium 官方 CNI 性能基准[58],Falco 2-5%
RSS 波动范围	60s 内 36.4-37.2MB（稳态 36.9MB）	—

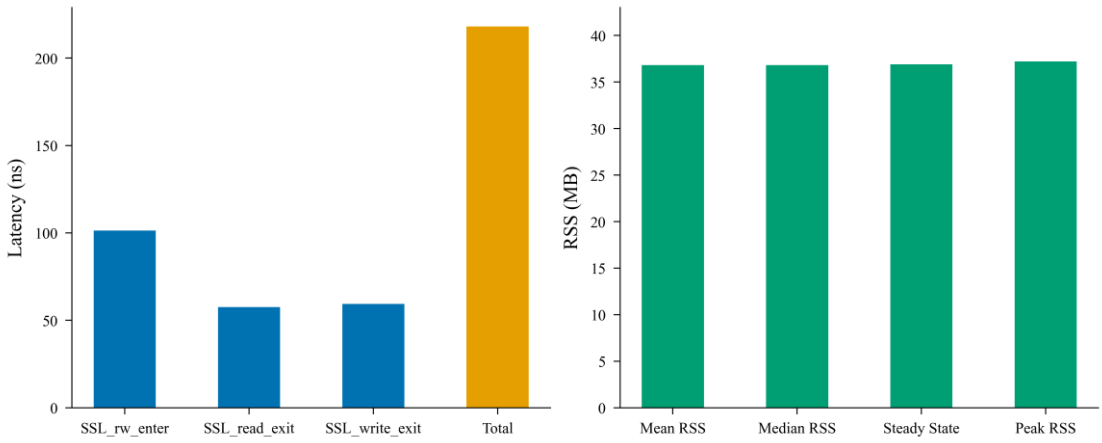


图 9　性能开销（左图为 SSL 探针延迟，右图为采集器 RSS 内存占用）

MtAgentLens 的低内存策略来自 per-CPU 环形缓冲区（256KB/CPU）的流式处理设计，事件在用户空间被及时消费，不在内核空间长期缓存。SSL 操作约 218ns 的额外延迟相对 TLS1.3 握手（1-5ms）占比低于 0.005%；稳态内存 36.9MB 低于 Falco（约 100MB）与 Tetragon（约 200MB）的公开工程经验值，60s 采样中的 RSS 位于 36.4-37.2MB 区间。端到端开销处于测量噪声范围内（Δ=-3.2%，标准差±0.01s）。RQ4 的结果表明，MtAgentLens 可以在常规 Linux Agent 运行环境中以较低资源成本提供持续监测。

7　讨论与未来工作

7.1　PBC对Type C的作用机制

Type C 攻击的关键特征是恶意目标通过多智能体协作过程展开。与单次 prompt 注入相比，这类攻击通常需要更多委派、上下文传递、文件读写和网络交互，因此更容易在 OS 层产生连续行为证据。

在本文基准中，子 Agent 委派会伴随进程创建和编排工具调用，任务交接会引入夹具文件或中间结果读写，命令控制和数据泄露则表现为外部网络连接或敏感路径访问。这些行为均可映射为策略边界穿越事件。良性任务也可能出现少量边界访问，但 Type C 场景中的平均 PBC 计数达到 396.7，是单 Agent 攻击的 5 至 7 倍。该差异解释了 MtAgentLens 在 Type C 场景中取得更高检测率的主要原因。

因此，PBC 的作用在于把跨 Agent 协作中难以由单个 prompt 表达的行为关系转化为可统计信号。责任图进一步将这些事件组织为可回溯路径，使检测结果能够关联到委派链、污点传播和敏感资源访问。消融实验中 L0→L1 的显著增益、Type C 的分类型结果以及案例中的 3-9 层因果链回溯，共同支持这一机制解释。

7.2　操作系统级可观测性的价值边界与部署策略

MtAgentLens 的优势区间明确：多智能体场景（Type C，PBC=396.7）中检测率达 1.000。在纯文本短格式场景（InjecAgent）中，MtAgentLens 与文本级基线相当（TPR=0.9934 vs Agent Sentinel 0.995），且 FPR 更低（0.2632 vs 0.3158）。

与运行时防御方法 Agent Sentinel[3]相比，MtAgentLens 的差异主要体现在两个方面。其一，信号源从文本或应用层交互扩展到 OS 级 eBPF 事件，在本文 Type C 场景中检测率由基线的 0.130 提升至 1.000。其二，分析单元从单智能体上下文扩展为跨智能体责任图，使深度 3 至 9 的因果链回溯成为可能。

基于上述分析，更合理的部署方式是分层防护。应用层检测器（如 WildGuard[9]）适合覆盖直接 prompt 注入、越狱文本和有害内容生成；MtAgentLens 则适合覆盖已经进入工具执行、文件处理、进程派生和网络通信阶段的行为风险。二者形成互补关系：应用层可以在执行前降低有害指令进入工作流的概率，操作系统层可以在执行后提供跨 Agent、跨进程和跨资源的行为证据。对于多智能体编排平台，建议将 PBC 计数、敏感资源访问和责任图回溯结果作为运行时审计信号，与提示级检测结果共同进入告警分级和事后调查流程。

7.3 校准模块的安全性考量

Observer 作为第三层语义仲裁模块，主要用于处理灰色地带场景。其设计遵循三个约束。第一，输入最小化：Observer 仅接收结构化因果链摘要、PBC 统计量和边类型计数，不直接接触完整原始 prompt 或文件内容。第二，触发受限：Observer 只在分数处于灰色地带、跨层增强、CUSUM 漂移或证据冲突时调用。第三，结果受信号约束：Observer 用于校准分数和确认灰区判定，不作为绕过 L1/L2 明确信号的独立检测器。

在 FuxiBench 上，Observer 共介入 39 个场景、产生 54 次调用；其中 Unsafe 上调 32 次，Safe 下调 16 次，Uncertain 4 次，Failed 2 次。触发原因以 grey_zone 为主（30 次），其次为 cross_layer_boost（15 次）和 cusum_drift（8 次）。这一分布说明，大部分场景可由 OS 信号和责任图直接判定，Observer 的主要价值在于分数校准和证据冲突处理。未来工作将进一步评估 Observer 的对抗鲁棒性，并探索基于可信执行环境的校准模块隔离方案。

表 12 汇总了 Observer 的调用次数、判定方向和触发原因分布。

表 12　Observer 调用与判定分布

类别	指标	数值	占 Observer 调用比例
调用次数	触发场景数	39	-
调用次数	Observer 调用总数	54	100.0%
判定方向	Unsafe 上调	32	59.3%
判定方向	Safe 下调	16	29.6%
判定方向	Uncertain	4	7.4%
判定方向	Failed	2	3.7%
输出判定	Unsafe	32	59.3%
输出判定	Safe	16	29.6%
输出判定	Uncertain	6	11.1%
触发原因	grey_zone	30	55.6%
触发原因	cross_layer_boost	15	27.8%
触发原因	cusum_drift	8	14.8%

类别	指标	数值	占 Observer 调用比例
触发原因	evidence_conflict	1	1.9%

7.4 总结与下一步研究方向

本文针对 LLM 多智能体系统中攻击行为跨越提示、进程、文件、网络和 Agent 协作关系的问题，提出了 MtAgentLens。该方法以零插桩 eBPF 探针采集 TLS、进程、文件和网络等 OS 级行为信号，通过策略边界穿越（PBC）量化刻画跨 Agent 协作中的敏感资源访问和委派行为，并利用责任图组织身份关联、污点传播和因果链回溯，结合六维行为评分与 Observer 校准实现安全监测。

在 FuxiBench v1.0.0 的 86 个冻结场景上，MtAgentLens 取得 ITT-TPR=0.896、PP-TPR=1.000、FPR=0.000 和 F1=0.945；在 Type C 多智能体协同攻击上达到 TPR=1.000，并支持深度 3-9 的因果链追踪。消融实验表明，加入 PBC 相关层后 AUROC 由 0.739 提升至 0.953，增益为 0.214；责任图进一步增强了证据组织和根因回溯能力，Observer 主要改善灰区分数校准。性能实验显示，系统单次 SSL 操作额外延迟约为 218ns，稳态内存占用为 36.9MB，端到端开销处于测量噪声范围内。

本文方法本质上是将多智能体协作过程中的 OS 级行为转化为可统计、可关联和可回溯的安全证据，为文本级防御难以覆盖的文件载荷、跨进程委派和多智能体协同攻击提供运行时监测路径。当前结果的适用范围主要由三方面界定：其一，系统面向 Linux/eBPF 执行环境，检测对象是已经产生 OS 级副作用的攻击；其二，FuxiBench 的场景规模和良性样本规模仍有限，需要更多工业工作流和对抗性场景进行验证；其三，低 OS 信号场景以及攻击者有意稀释或规避 PBC 行为时，系统需要与提示级检测方法协同工作。

下一步将重点开展三方面研究：一是扩展 FuxiBench 的场景规模，增加低信号攻击、困难良性样本和跨框架多智能体工作流；二是构建 OS 行为信号与 prompt 语义信号的联合评分和在线校准机制，提升低信号场景下的检测能力；三是研究面向 PBC 规避、事件噪声和跨平台部署的鲁棒监测方法，并探索 Windows ETW、macOS Endpoint Security 等替代观测机制。

References:

[2] LangChain. LangGraph overview[EB/OL]. 2026. URL: https://docs.langchain.com/oss/python/langgraph/overview.

[3] Hu H, Chen P, Zhao Y, Chen Y. Agentsentinel: an end-to-end and real-time security defense framework for computer-use agents. Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security. 2025: 3535～3549.

[4] Zhang J, Yang S, Li B. Udora: a unified red teaming framework against llm agents by dynamically hijacking their own reasoning. arXiv preprint arXiv:2503.01908, 2025.

[5] Chang H, Jun Y, Lee H. Chatinject: abusing chat templates for prompt injection in llm agents. International Conference on Learning Representations. 2026, 2026: 24688～24715.

[6] Chennabasappa S, Nikolaidis C, Song D, Molnar D, Ding S, Wan S, Whitman S, Deason L, Doucette N, Montilla A, Others. Llamafirewall: an open source guardrail system for building secure ai agents. arXiv preprint arXiv:2505.03574, 2025.

[7] Meta Llama. Llama Prompt Guard 2 86M model card[EB/OL]. 2026. URL: https://raw.githubusercontent.com/meta-llama/PurpleLlama/main/Llama-Prompt-Guard-2/86M/MODEL_CARD.md.

[8] Protect AI. LLM Guard: the security toolkit for LLM interactions[EB/OL]. 2026. URL: https://protectai.github.io/llm-guard/.

[9] Han S, Rao K, Ettinger A, Jiang L, Lin BY, Lambert N, Choi Y, Dziri N. Wildguard: open one-stop moderation tools for safety risks, jailbreaks, and refusals of llms. Advances in neural information processing systems, 2024, 37: 8093～8131.

[10] Chen S, Piet J, Sitawarin C, Wagner D. {StruQ}: defending against prompt injection with structured queries. 34th USENIX Security Symposium (USENIX Security 25). 2025: 2383～2400.

[11] Chen S, Zharmagambetov A, Mahloujifar S, Chaudhuri K, Wagner D, Guo C. Secalign: defending against prompt injection with preference optimization. Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security. 2025: 2833～2847.

[12] Chen S, Zharmagambetov A, Wagner D, Guo C. Meta secalign: a secure foundation llm against prompt injection attacks. arXiv preprint arXiv:2507.02735, 2025.

[13] Waknin L, Levi YY, Solomon R, Kotak J, Giloni A, Picardi C, Vainshtein R, Elovici Y, Shabtai A. MAStitch: unifying local and global perspectives for anomaly detection in multi-agent systems. Proc. of the 25th International Conference on Autonomous Agents and Multiagent Systems. 2026: 1874～1882.

[14] Miao R, Liu Y, Wang Y, Shen X, Tan Y, Dai Y, Pan S, Wang X. Blindguard: safeguarding llm-based multi-agent systems under unknown attacks. Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). 2026: 39215～39234.

[15] Zhou J, Wang L, Yang X. Guardian: safeguarding llm multi-agent collaborations with temporal graph modeling. Advances in Neural Information Processing Systems, 2026, 38: 7973～8001.

[18] Linux Kernel Organization. BPF documentation[EB/OL]. 2026. URL: https://www.kernel.org/doc/html/latest/bpf/index.html.

[23] Pham L, Ha H, Zhang H. Baro: robust root cause analysis for microservices via multivariate bayesian online change point detection. Proceedings of the ACM on Software Engineering, ACM New York, NY, USA, 2024, 1(FSE): 2214～2237.

[24] Zhan Q, Liang Z, Ying Z, Kang D. Injecagent: benchmarking indirect prompt injections in tool-integrated large language model agents. Findings of the Association for Computational Linguistics: ACL 2024. 2024: 10471～10506.

[25] Liu X, Xu N, Chen M, Xiao C. Autodan: generating stealthy jailbreak prompts on aligned large language models. International Conference on Learning Representations. 2024, 2024: 56174～56194.

[26] Mehrotra A, Zampetakis M, Kassianik P, Nelson B, Anderson H, Singer Y, Karbasi A. Tree of attacks: jailbreaking black-box llms automatically. Advances in Neural Information Processing Systems, 2024, 37: 61065～61105.

[27] Zou A, Wang Z, Carlini N, Nasr M, Kolter JZ, Fredrikson M. Universal and transferable adversarial attacks on aligned language models. arXiv preprint arXiv:2307.15043, 2023.

[28] Andriushchenko M, Souly A, Dziemian M, Duenas D, Lin M, Wang J, Hendrycks D, Zou A, Kolter Z, Fredrikson M, Others. Agentharm: a benchmark for measuring harmfulness of llm agents. International Conference on Learning Representations. 2025, 2025: 79185～79220.

[29] Liu Y, Jia Y, Geng R, Jia J, Gong NZ. Formalizing and benchmarking prompt injection attacks and defenses. 33rd USENIX Security Symposium (USENIX Security 24). 2024: 1831～1847.

[30] Kim J, Guo W, Song D, Berkeley U, Santa Barbara U. Sok: attack and defense landscape of agentic ai systems. 35nd USENIX Security Symposium (USENIX Security 26). 2026.

[31] Xu Q, Tian Z, Wu H, Huang Z, Liang D, Song Y, Pan Z, Liu F, Li D. Disguise while defense: avoid refusal responses in llm’s defense via a multi-agent attacker‐disguiser game. IEEE Transactions on Information Forensics and Security, IEEE, 2025, 21: 1006～1020.

[32] Huang Z, Liu Z, Luo M, Wu W, Cai Z. Propagating unsafe actions in llm controlled multi-robot collaboration via single robot compromise. arXiv preprint arXiv:2605.15641, 2026.

[33] Maiti S. Caging the agents: a zero trust security architecture for autonomous ai in healthcare. arXiv preprint arXiv:2603.17419, 2026.

[34] Lin H, Lao Y, Geng T, Yu T, Zhao W. Uniguardian: a unified defense for detecting prompt injection, backdoor attacks and adversarial attacks in large language models. arXiv preprint arXiv:2502.13141, 2025.

[35] Debenedetti E, Zhang J, Balunović M, Beurer-Kellner L, Fischer M, Tramèr F. AgentDojo: a dynamic environment to evaluate attacks and defenses for llm agents. arXiv preprint arXiv:2406.13352, 2024.

[36] Abramovich T, Udeshi M, Shao M, Lieret K, Xi H, Milner K, Jancheska S, Yang J, Jimenez CE, Khorrami F, Others. EnIGMA: interactive tools substantially assist lm agents in finding security vulnerabilities. arXiv preprint arXiv:2409.16165, 2024.

[37] Scheurer J, Balesni M, Hobbhahn M. Large language models can strategically deceive their users when put under pressure. arXiv preprint arXiv:2311.07590, 2023.

[38] Greshake K, Abdelnabi S, Mishra S, Endres C, Holz T, Fritz M. Not what you've signed up for: compromising real-world llm-integrated applications with indirect prompt injection. Proceedings of the 16th ACM workshop on artificial intelligence and security. 2023: 79～90.

[39] Robey A, Wong E, Hassani H, Pappas GJ. Smoothllm: defending large language models against jailbreaking attacks. arXiv preprint arXiv:2310.03684, 2023.

[40] Cilium. Tetragon overview[EB/OL]. 2026. URL: https://tetragon.io/docs/overview/.

[41] Falco Project. The Falco project: cloud native runtime security[EB/OL]. 2026. URL: https://falco.org/docs/.

[42] Zheng Y, Hu Y, Yu T, Quinn A. Agentsight: system-level observability for ai agents using ebpf. Proceedings of the 4th Workshop on Practical Adoption Challenges of ML for Systems. 2025: 110～115.

[43] Lu H, Wang S, Wu Y, He W, Zhang F. {MOAT}: towards safe {bpf} kernel extension. 33rd USENIX Security Symposium (USENIX Security 24). 2024: 1153～1170.

[44] Zhang P, Wu C, Meng X, Zhang Y, Peng M, Zhang S, Hu B, Xie M, Lai Y, Kang Y, Others. {HIVE}: a hardware-assisted isolated execution environment for {ebpf} on {aarch64}. 33rd USENIX Security Symposium (USENIX Security 24). 2024: 163～180.

[45] Patel J, Buhl-Nielsen LG, Ghosn A, Kogias M. {KRAKENGUARD}: towards {fine-grained}{ebpf} isolation. 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26). 2026: 2685～2704.

[46] He Y, Guo R, Xing Y, Che X, Sun K, Liu Z, Xu K, Li Q. Cross container attacks: the bewildered {ebpf} on clouds. 32nd USENIX Security Symposium (USENIX Security 23). 2023: 5971～5988.

[47] Zhang G, Wang J, Chen J, Zhou W, Wang K, Yan S. Agentracer: who is inducing failure in the llm agentic systems? International Conference on Learning Representations. 2026, 2026: 11377～11399.

[48] Zeng A, Liu M, Lu R, Wang B, Liu X, Dong Y, Tang J. Agenttuning: enabling generalized agent abilities for llms. Findings of the Association for Computational Linguistics: ACL 2024. 2024: 3053～3077.

[49] Kazari K, Shereen E, Dán G. Decentralized anomaly detection in cooperative multi-agent reinforcement learning. IJCAI. 2023: 162～170.

[50] Zhou Y, Wang Z, Dharanipragada S, Yu M. Electrode: accelerating distributed protocols with {ebpf}. 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23). 2023: 1391～1407.

[51] Pham L, Ha H, Zhang H. Root cause analysis for microservice system based on causal inference: how far are we? Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering. 2024: 706～715.

[52] Yao Z, Pei C, Chen W, Wang H, Su L, Jiang H, Xie Z, Nie X, Pei D. Chain-of-event: interpretable root cause analysis for microservices through automatically learning weighted event causal graph. Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering. 2024: 50～61.

[57] DeLong E R, DeLong D M, Clarke-Pearson D L. Comparing the areas under two or more correlated receiver operating characteristic curves: a nonparametric approach. Biometrics, 1988, 44(3): 837～845.

[58] Cilium. CNI performance benchmark[EB/OL]. 2026. URL: https://docs.cilium.io/en/stable/operations/performance/benchmark/.

附中文参考文献:

[1] 王路桥, 周洋涛, 李青山, 王铭康, 徐子轩, 崔笛, 王璐, 罗懿行. 基于大语言模型的多智能体协作代码评审人推荐. 软件学报, 2024, 36(6): 2558～2575.

[16] 台建玮, 杨双宁, 王佳佳, 李亚凯, 刘奇旭, 贾晓启. 大语言模型对抗性攻击与防御综述. 计算机研究与发展, 2025, 62(3): 563～588.

[17] 李南, 丁益东, 江浩宇, 牛佳飞, 易平. 面向大语言模型的越狱攻击综述. 计算机研究与发展, 2024, 61(5): 1156～1181.

[19] 李亚晓, 李青山, 王璐, 姜宇轩. AmazeMap: 基于多层次影响图的微服务故障定位方法. 软件学报, 2024, 35(7): 3115～3140.

[20] 于庆洋, 白晓颖, 李明杰, 李奇原, 刘涛, 刘泽胤, 裴丹. 基于调用链控制流分析的大型微服务系统性能建模与异常定位. 软件学报, 2022, 33(5): 1849～1864.

[21] 靖宇涵, 何波, 张凌昕, 李天星, 王敬宇, 刘聪. PASER: 加性多维 kpi 异常根因定位模型. 软件学报, 2021, 33(2): 738～750.

[22] 孙德权, 周竞文, 周海芳. SOTIMiner: 一种基于集合运算的时序不变式挖掘方法. 软件学报, 2022, 33(2): 455～472.
[53] 李晓卓, 卿笃军, 贺也平, 马恒太. 基于源代码扩展信息的细粒度缺陷定位方法. 软件学报, 2022, 33(11): 4008～4026.
[54] 包航宇, 殷康璘, 曹立, 李世宁, 孙永敬, 尹汇锋, 汤汝鸣, 侯岳, 王士强, 裴丹, 杨晓勤, 王立新. 智能运维的实践:现状与标准化. 软件学报, 2023, 34(9): 4069～4095.
[55] 殷康璘, 杜庆峰. 基于混沌工程的微服务韧性风险识别和分析. 软件学报, 2021, 32(5): 1231～1255.
[56] 张明悦, 金芝, 刘坤. 合作-竞争混合型多智能体系统的虚拟遗憾优势自博弈方法. 软件学报, 2023, 35(2): 739～757.

作者简介

唐浩钺, 硕博连读生, 主要研究领域为系统软件、智能软件、分布式计算.
刘周康, 本科生, 主要研究领域为操作系统、智能软件, 多次获学业优异奖学金，在国际多次、全国学科竞赛中获奖，开源软件项目开发及维护者.
任怡, 博士, 研究员，博士生导师, CCF 杰出会员, 主要研究领域为系统软件、云边协同、智能软件、分布式计算.
张建锋, 博士, 副研究员，硕士生导师, 主要研究领域为智能基础软件、系统安全、分布式计算.
赵欣 , 博士, 助理研究员，主要研究领域为智能基础软件、分布式计算.
马俊, 博士, 副研究员，硕士生导师, 主要研究领域为操作系统、系统安全、智能软件.
余杰, 博士, 研究员，博士生导师, CCF 杰出会员, 主要研究领域为系统软件、人工智能.